

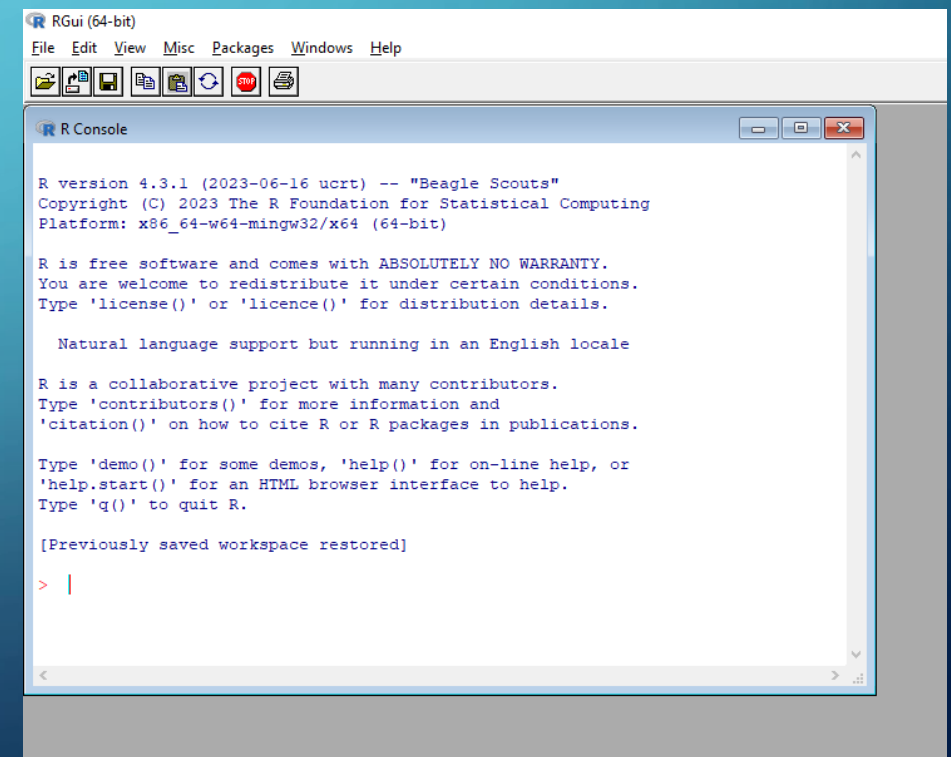
The background is a gradient of blue shades, from a lighter cyan at the top to a darker blue at the bottom. A large, semi-transparent black rectangle is centered horizontally and vertically. On the left and right sides of this rectangle, there are stylized white circuit lines with small circles at the ends, resembling a printed circuit board. The main title is written in white, bold, uppercase letters inside the black rectangle.

INTRODUCTION TO R

WORKING WITH RSTUDIO

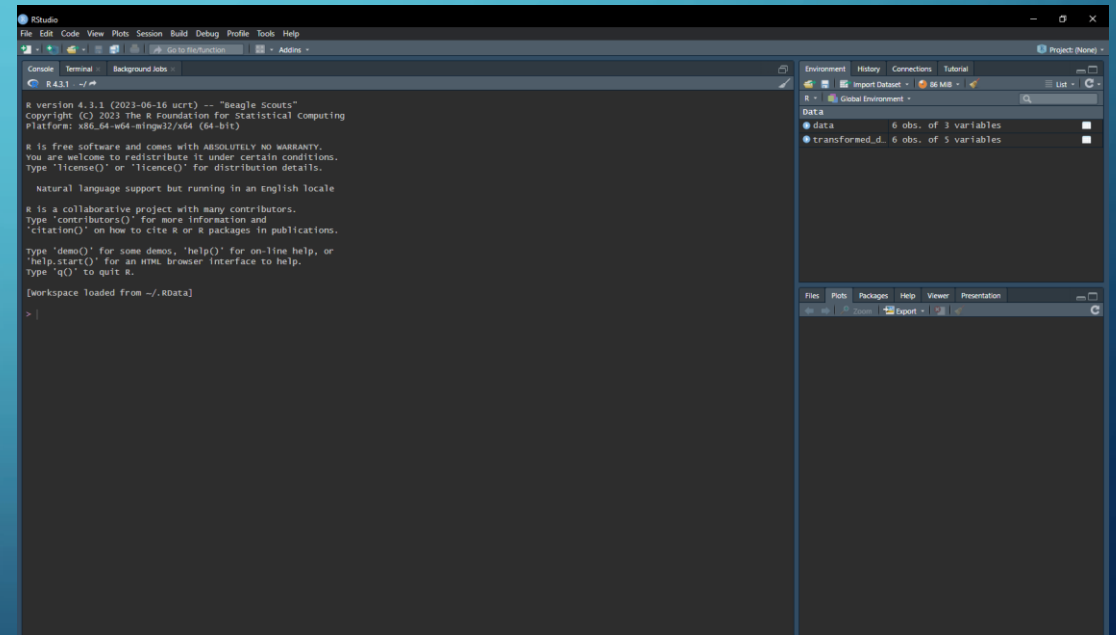
WHAT IS R?

- Language and environment for statistical computing and graphics
- Free and open-source
- Runs on Windows, MacOS, and Linux
- Packages (pre-built functions/code) created by the community available for specific use cases
 - Comprehensive R Archive Network (CRAN) currently has 20,828 packages available (<https://cran.r-project.org/web/packages/>)
 - Other niche repositories exist, e.g. BioConductor for bioinformatics



WHAT IS RSTUDIO?

- Integrated development environment (IDE) for R.
- More user-friendly than “base” R
- Runs on Windows, MacOS, and Linux
- We will be using RStudio which is available on the TSU BioRACC Cluster
- You can also install R/RStudio locally on your laptop/desktop



1: Install R

RStudio requires R 3.6.0+. Choose a version of R that matches your computer's operating system.

DOWNLOAD AND INSTALL R

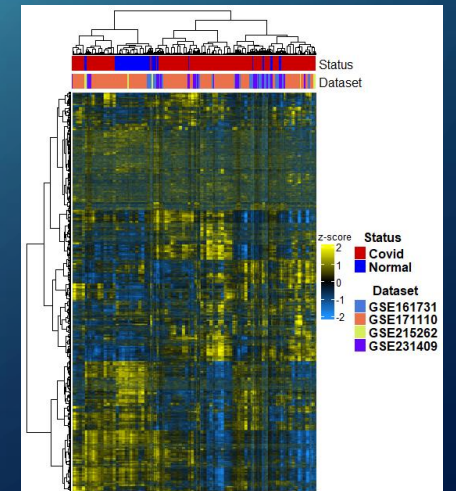
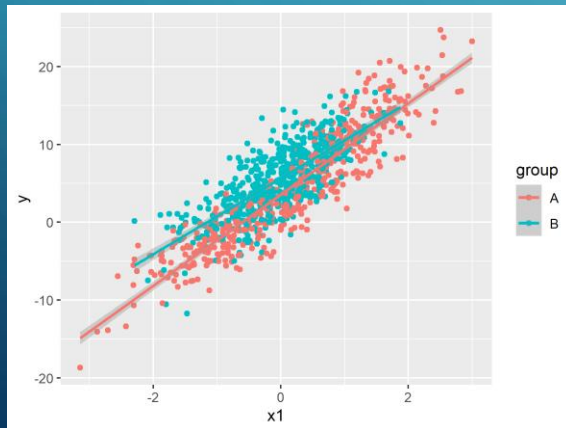
2: Install RStudio

[DOWNLOAD RSTUDIO DESKTOP FOR WINDOWS](#)

Size: 263.07 MB | [SHA-256: 44C8797C](#) | Version: 2024.04.1+748 |
Released: 2024-05-11

WHY R FOR BIOINFORMATICS?

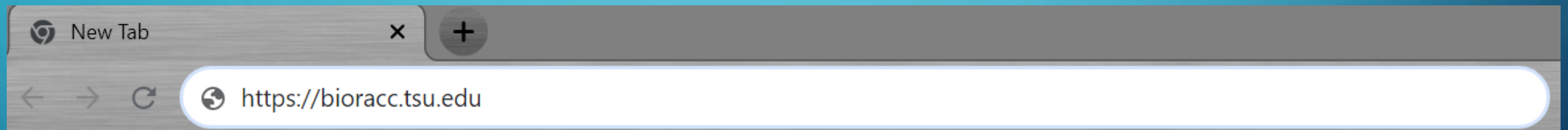
- Statistics – for large “omics” datasets
- Data wrangling – for large “omics” datasets
- Bioconductor – repository of packages specifically for biological data analysis
- Visualization – ggplot2 package for publication-ready figures



LOGGING ON TO THE CLUSTER

Type **`https://bioracc.tsu.edu`** in a browser

(Google Chrome preferably)



LOGGING ON

Username: **firstname.lastname** (all lowercase)

Default password: **firstname.lastname .changeme**

*Did you change your password last time?

Sign in to access this site
Authorization required by <https://bioracc.tsu.edu>

Username

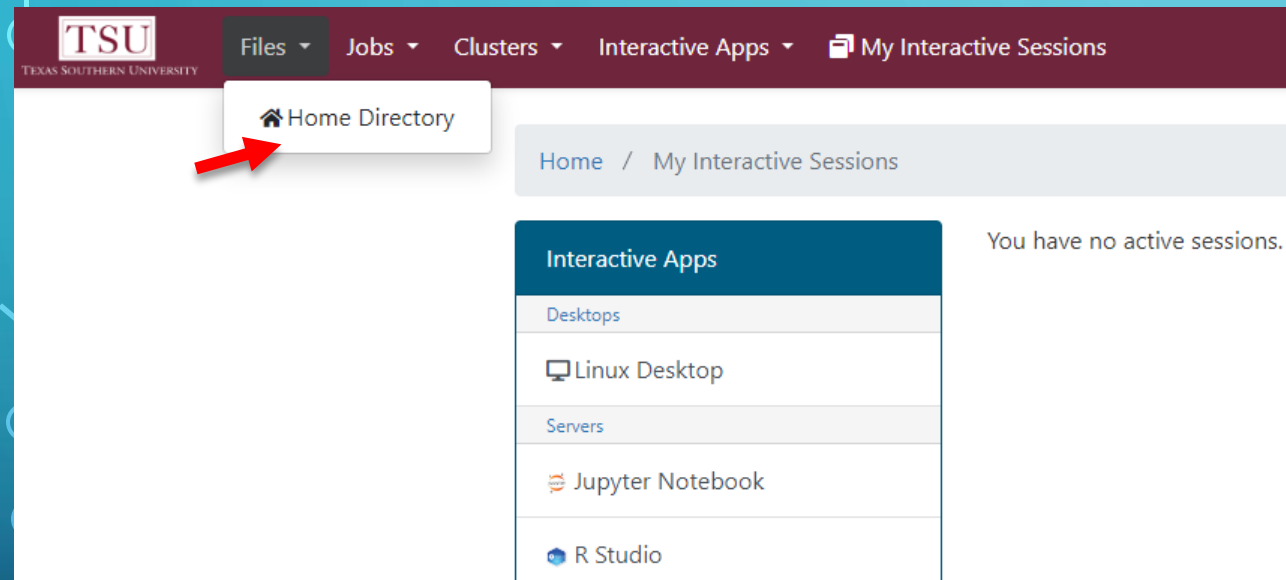
Password

Sign in

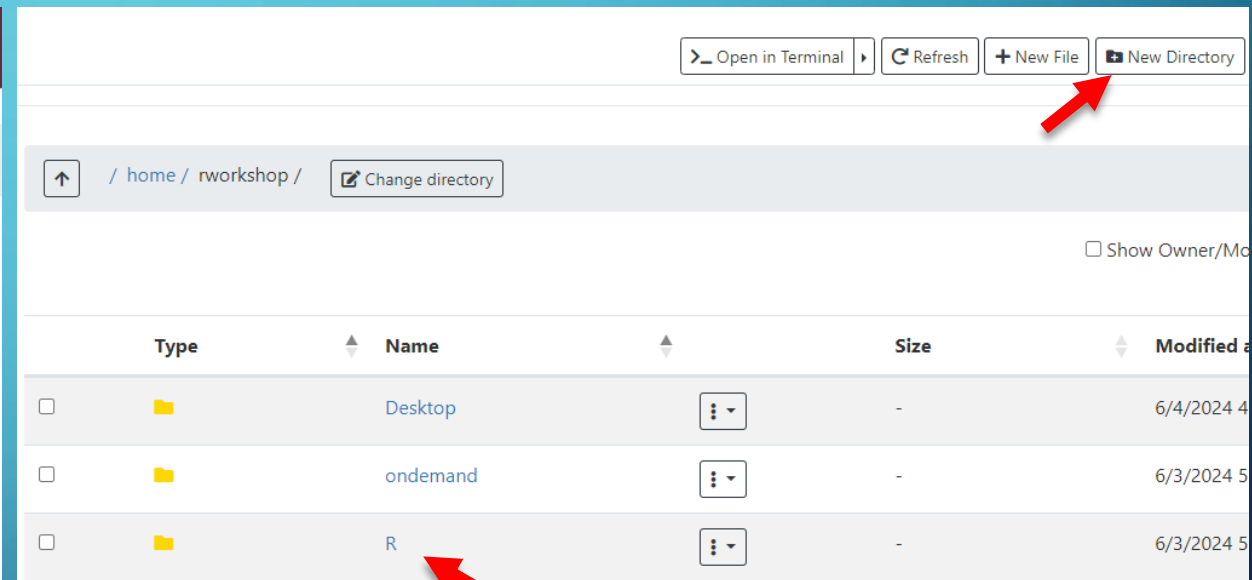
Cancel

BE ORGANIZED

First, create a directory within your home directory called “R”



The screenshot shows the top navigation bar of the TSU portal with links for Files, Jobs, Clusters, Interactive Apps, and My Interactive Sessions. A red arrow points to the 'Files' dropdown menu, which is open and shows 'Home Directory' as the selected option. Below the navigation bar, the breadcrumb path is 'Home / My Interactive Sessions'. On the left, there is a sidebar with 'Interactive Apps' including Desktops (Linux Desktop), Servers (Jupyter Notebook), and R Studio. The main content area states 'You have no active sessions.'



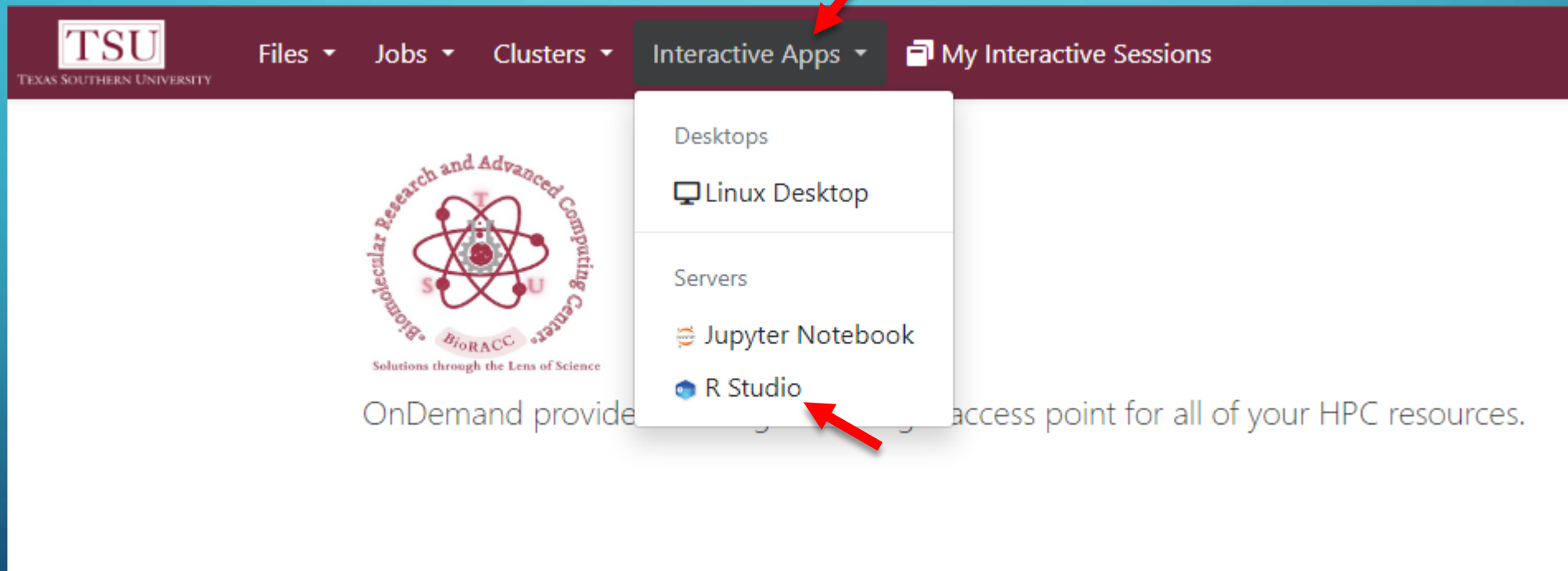
The screenshot shows the file manager interface. At the top, there are buttons for 'Open in Terminal', 'Refresh', '+ New File', and '+ New Directory'. A red arrow points to the '+ New Directory' button. Below the buttons, the breadcrumb path is '/ home / rworkshop /' and there is a 'Change directory' button. A table lists the contents of the directory:

	Type	Name	Size	Modified
☐	Folder	Desktop	-	6/4/2024 4
☐	Folder	ondemand	-	6/3/2024 5
☐	Folder	R	-	6/3/2024 5

A red arrow points to the 'R' directory in the table.

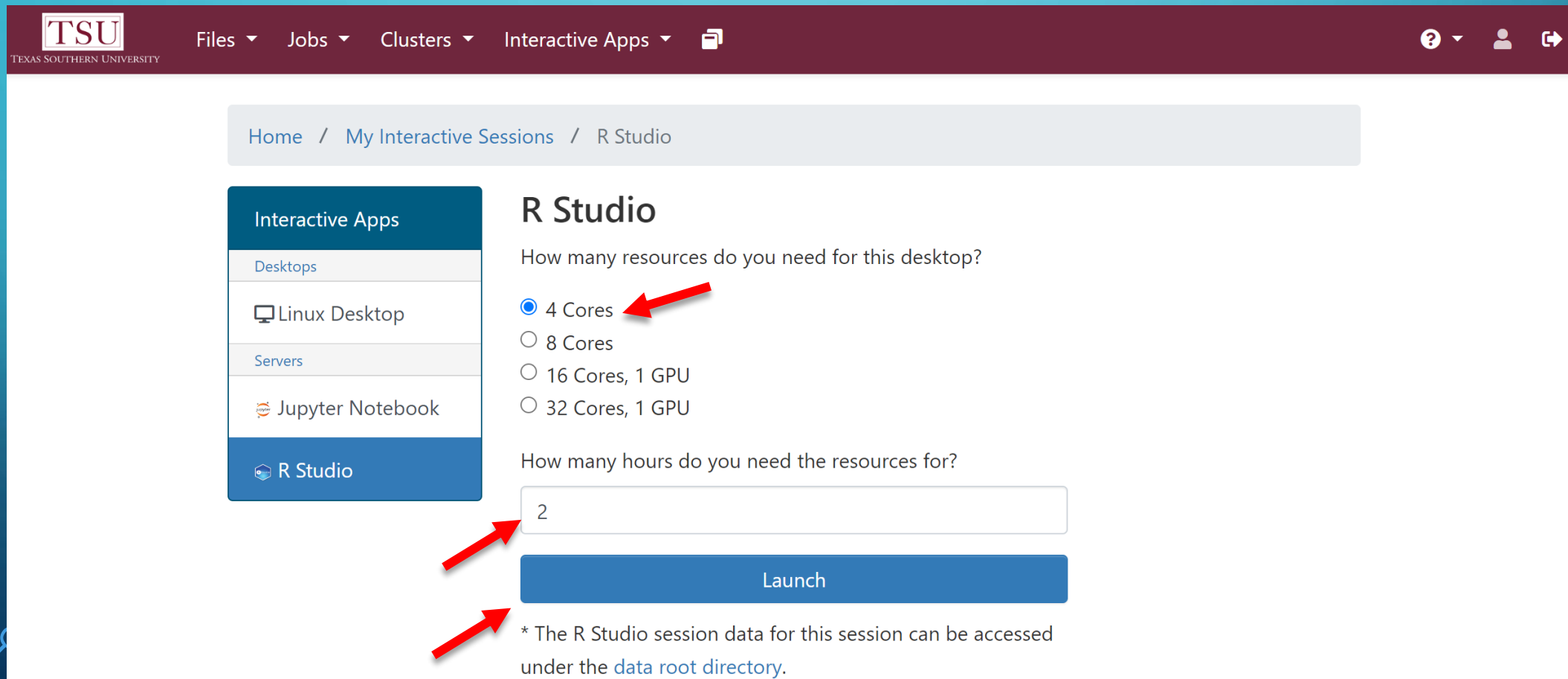
REQUESTING AN RSTUDIO SESSION

Interactive Apps → R Studio



REQUESTING AN RSTUDIO SESSION

Select **4 Cores** and enter **2** for the number of hours, then click Launch



The screenshot shows the Texas Southern University (TSU) Interactive Apps interface. The top navigation bar includes the TSU logo, a search bar, and links for Files, Jobs, Clusters, and Interactive Apps. The main content area is titled 'R Studio' and contains a form for requesting a session. On the left, a sidebar lists 'Interactive Apps' with options for Desktops (Linux Desktop), Servers (Jupyter Notebook), and R Studio. The R Studio section is selected. The form asks 'How many resources do you need for this desktop?' with radio button options: 4 Cores (selected), 8 Cores, 16 Cores, 1 GPU, and 32 Cores, 1 GPU. A red arrow points to the '4 Cores' option. Below this, it asks 'How many hours do you need the resources for?' with a text input field containing the number '2'. A red arrow points to the input field. At the bottom, there is a blue 'Launch' button and a note: '* The R Studio session data for this session can be accessed under the [data root directory](#).' A red arrow points to the 'Launch' button.

TSU
TEXAS SOUTHERN UNIVERSITY

Files ▾ Jobs ▾ Clusters ▾ Interactive Apps ▾

Home / My Interactive Sessions / R Studio

Interactive Apps

- Desktops
- Linux Desktop
- Servers
- Jupyter Notebook
- R Studio**

R Studio

How many resources do you need for this desktop?

☒ 4 Cores

☐ 8 Cores

☐ 16 Cores, 1 GPU

☐ 32 Cores, 1 GPU

How many hours do you need the resources for?

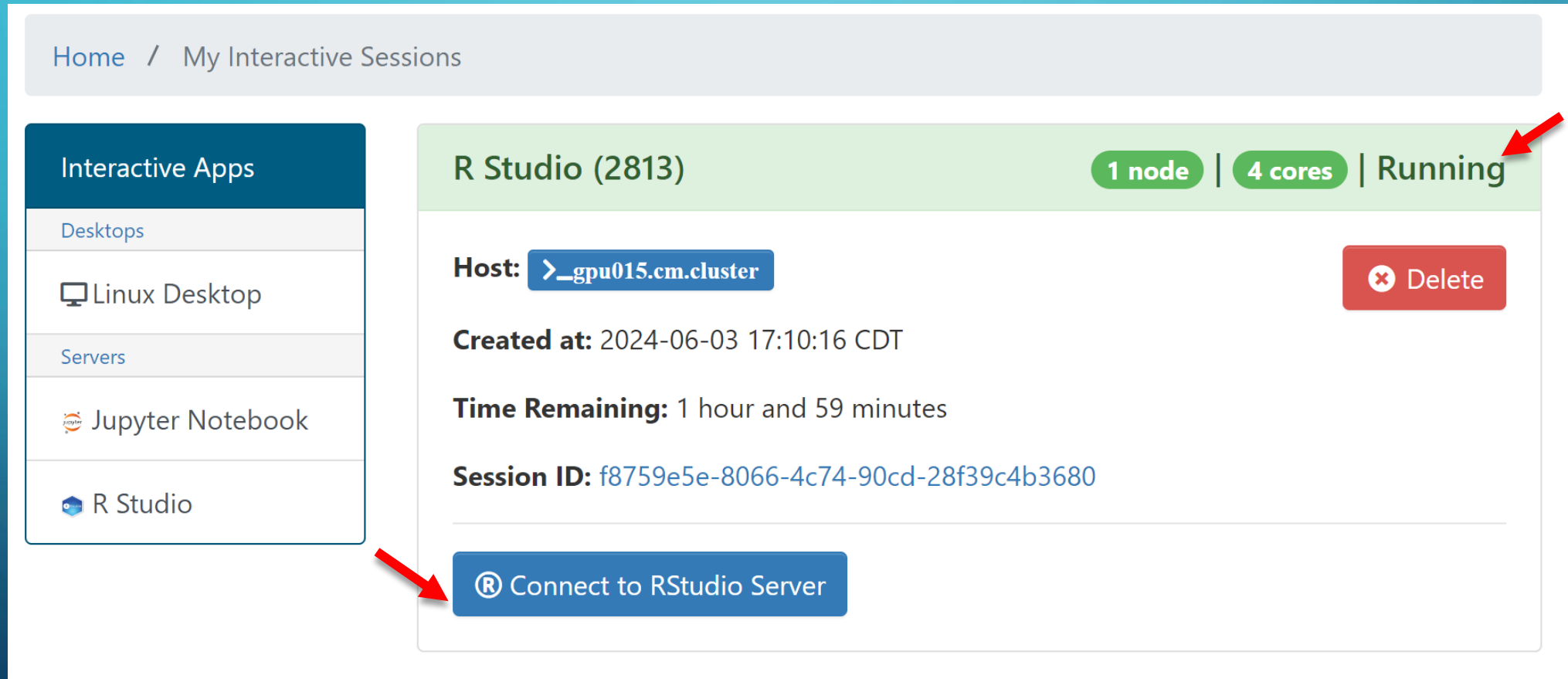
2

Launch

* The R Studio session data for this session can be accessed under the [data root directory](#).

REQUESTING AN RSTUDIO SESSION

Click *Connect to Rstudio Server* when session is running



The screenshot displays a web interface for managing interactive sessions. At the top, a breadcrumb trail shows 'Home / My Interactive Sessions'. On the left, a sidebar titled 'Interactive Apps' contains a list of available applications: 'Desktops', 'Linux Desktop', 'Servers', 'Jupyter Notebook', and 'R Studio'. The 'R Studio' option is selected. The main content area shows details for an active session named 'R Studio (2813)'. The session status is 'Running', with '1 node' and '4 cores' allocated. A red arrow points to the 'Running' status. Below the session name, the 'Host' is listed as '>_gpu015.cm.cluster', and a red 'Delete' button is visible. The 'Created at' timestamp is '2024-06-03 17:10:16 CDT', and the 'Time Remaining' is '1 hour and 59 minutes'. The 'Session ID' is 'f8759e5e-8066-4c74-90cd-28f39c4b3680'. At the bottom of the session details, a blue button labeled '® Connect to RStudio Server' is highlighted with a red arrow.

Home / My Interactive Sessions

Interactive Apps

- Desktops
- Linux Desktop
- Servers
- Jupyter Notebook
- R Studio

R Studio (2813) 1 node | 4 cores | Running

Host: >_gpu015.cm.cluster Delete

Created at: 2024-06-03 17:10:16 CDT

Time Remaining: 1 hour and 59 minutes

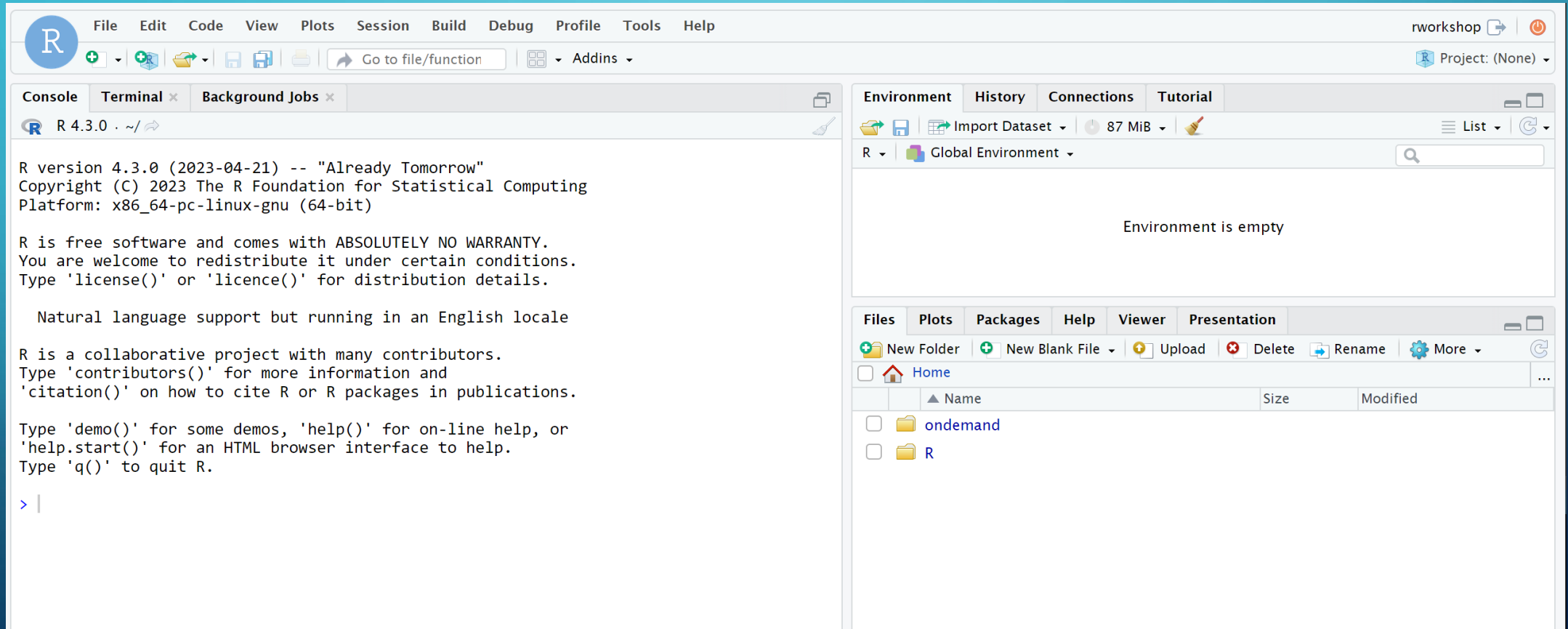
Session ID: f8759e5e-8066-4c74-90cd-28f39c4b3680

® Connect to RStudio Server

RSTUDIO SESSION



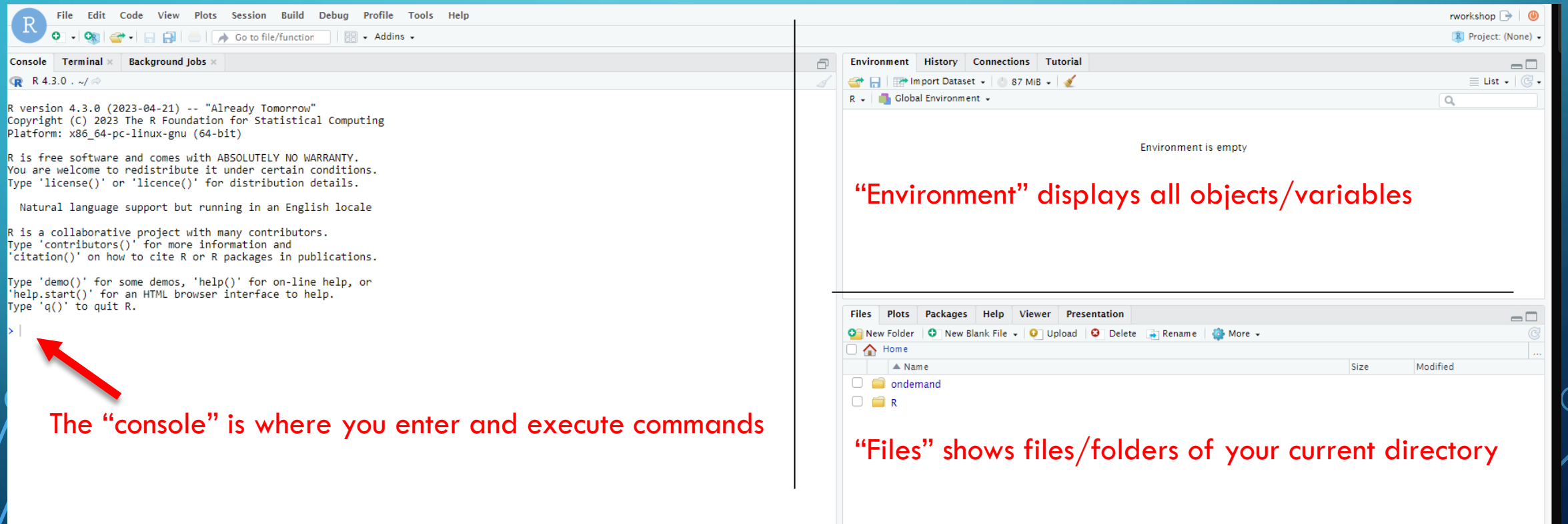
A new browser window will open with your RStudio session in your home directory



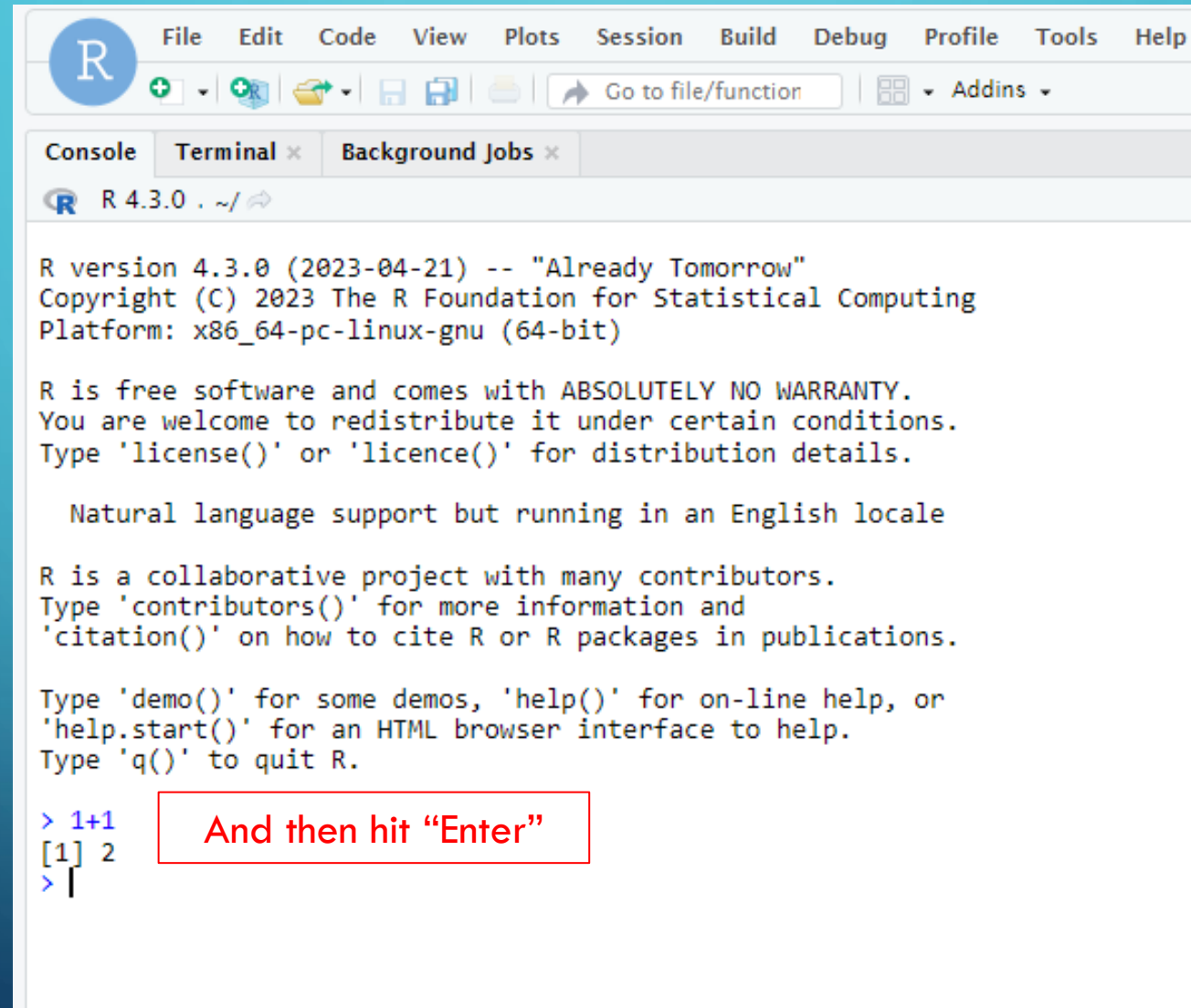
The screenshot displays the RStudio application window. The top menu bar includes File, Edit, Code, View, Plots, Session, Build, Debug, Profile, Tools, and Help. The top right shows the user 'rworkshop' and a project status of 'Project: (None)'. The main interface is divided into four panes:

- Console:** Shows the R version 4.3.0 (2023-04-21) and the standard startup message. The prompt is currently at the first line: `> |`.
- Environment:** Displays 'Global Environment' and indicates that the environment is empty.
- Files:** Shows the file explorer for the home directory, listing folders 'ondemand' and 'R'.
- Plots/Packages/Help/Viewer/Presentation:** These panes are currently empty.

RSTUDIO LAYOUT



CONSOLE



The screenshot shows the R console window with the following content:

```
R version 4.3.0 (2023-04-21) -- "Already Tomorrow"
Copyright (C) 2023 The R Foundation for Statistical Computing
Platform: x86_64-pc-linux-gnu (64-bit)

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

Natural language support but running in an English locale

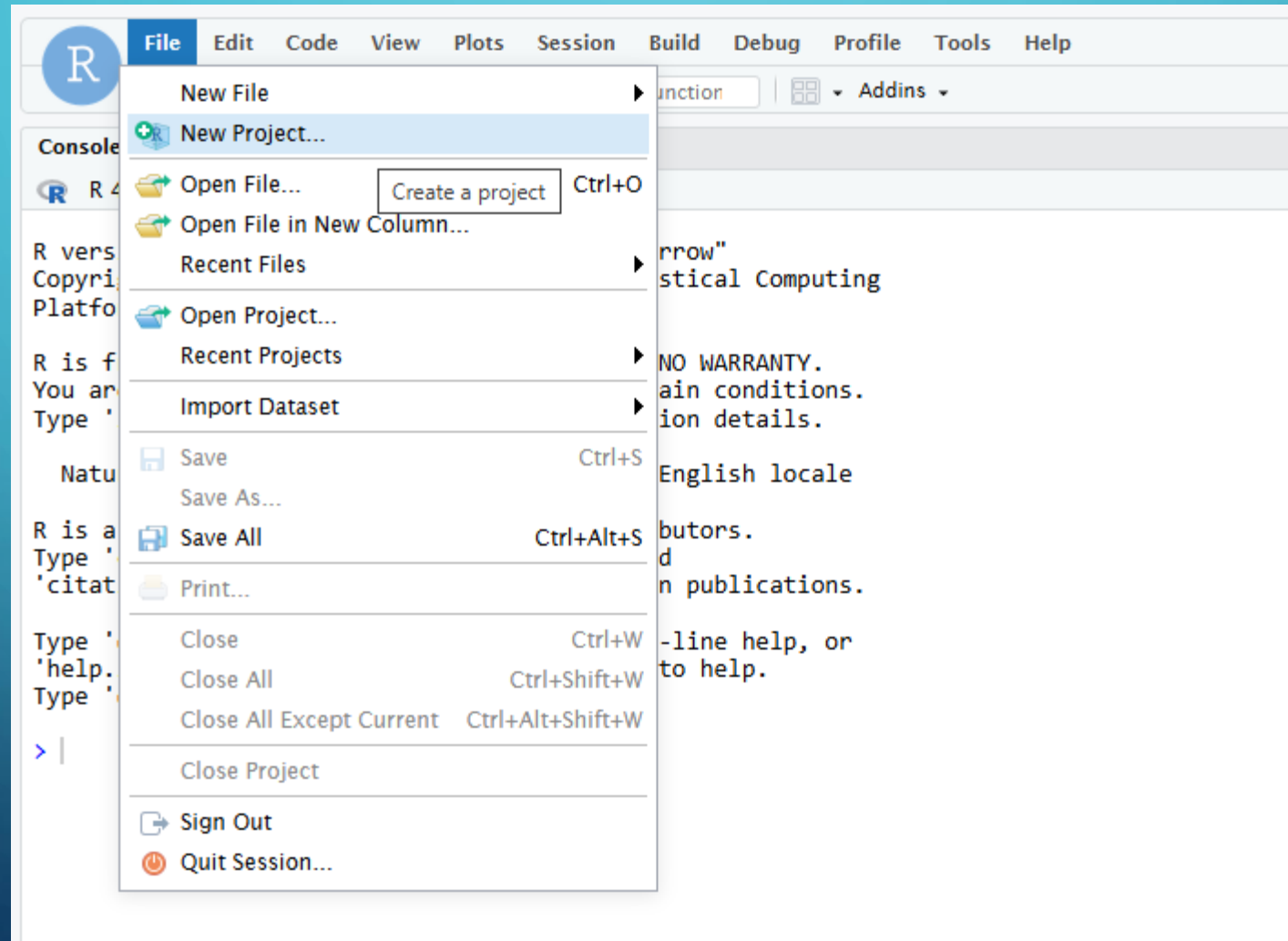
R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

> 1+1
[1] 2
> |
```

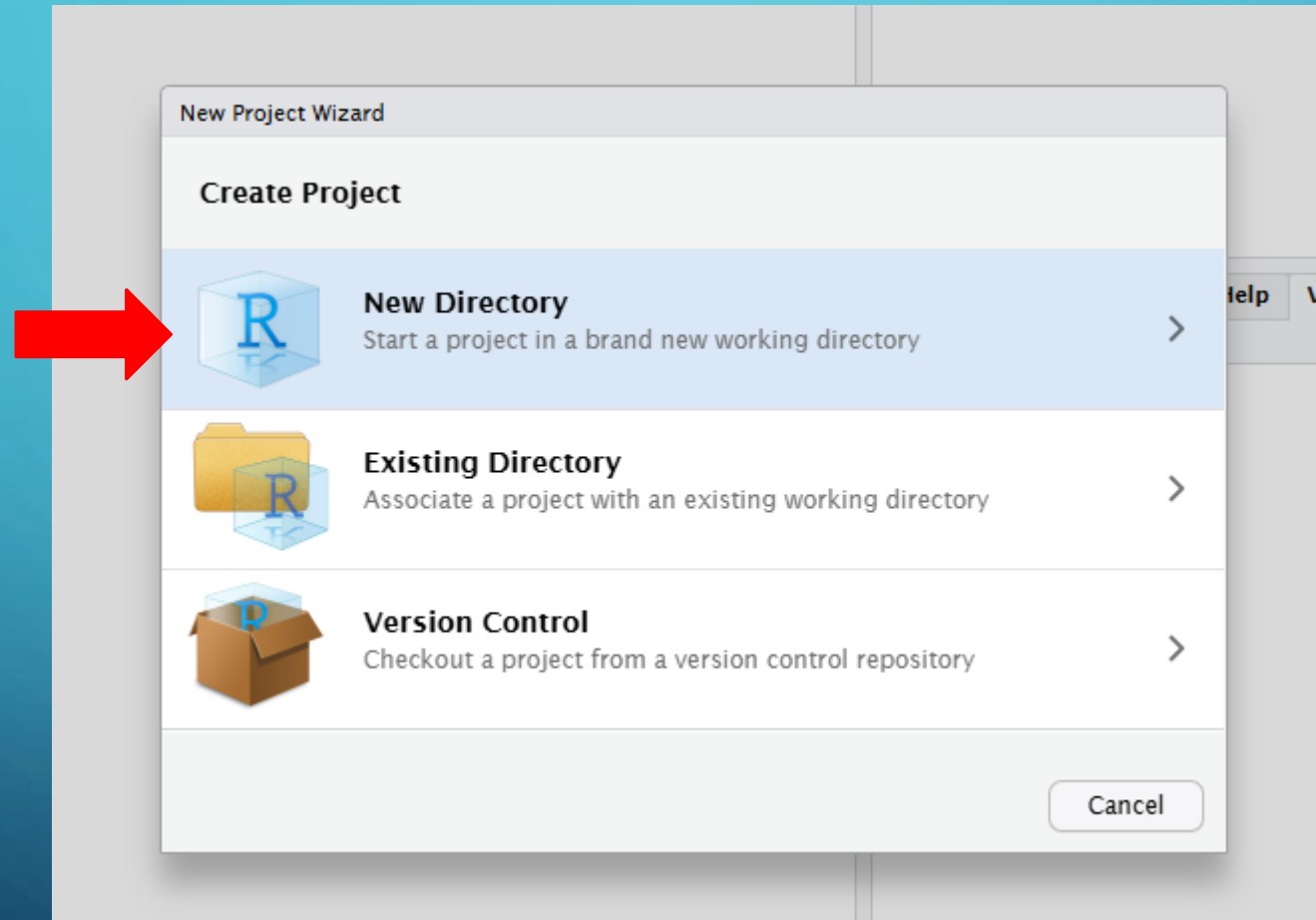
A red box highlights the text "And then hit 'Enter'" next to the prompt "> |".

CREATING A PROJECT

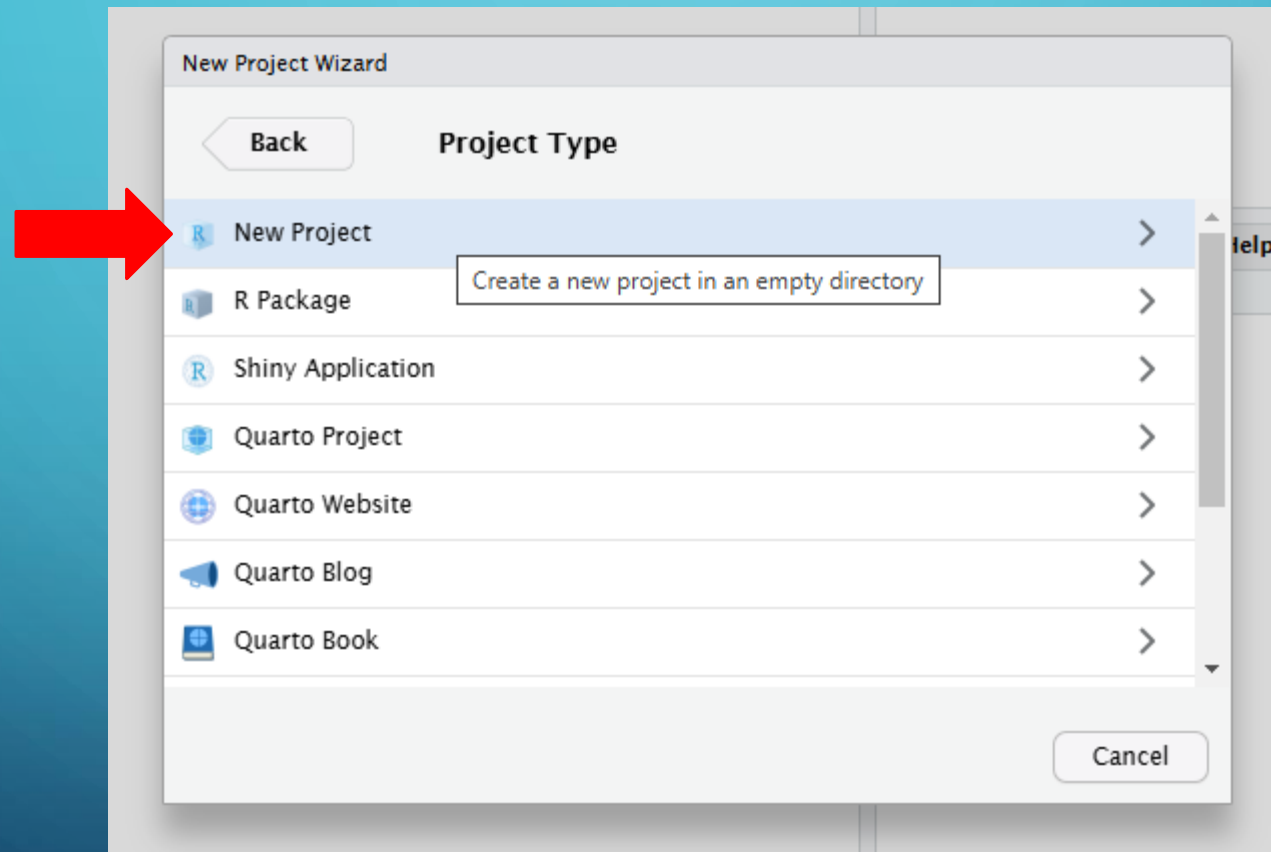


Projects are a good way to stay organized

CREATING A PROJECT



CREATING A PROJECT



CREATING A PROJECT

New Project Wizard

Back Create New Project

Directory name:
intro

Create project as subdirectory of:
~/R

☐ Use renv with this project

Create Project Cancel

Choose Directory

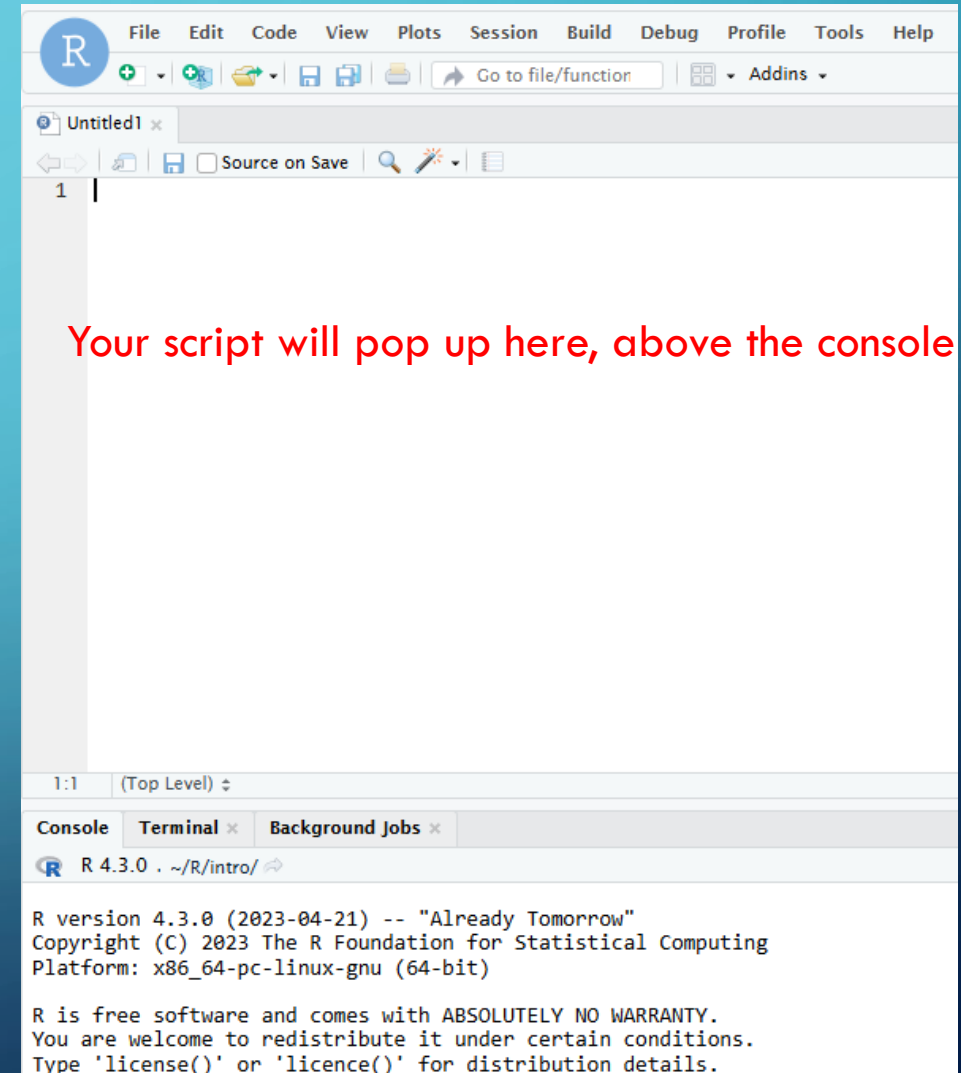
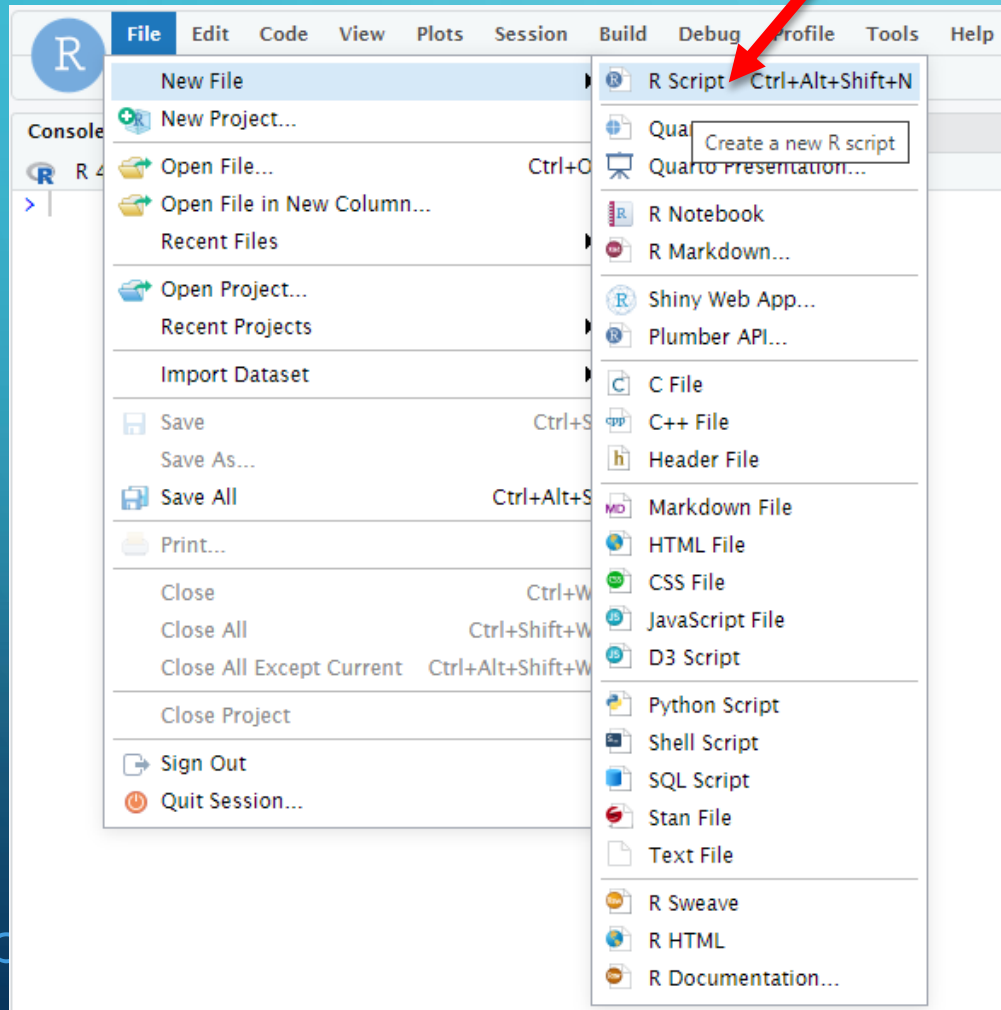
Folder: ~/R

- Home
- Desktop
- gsea_home
- linux
- ondemand
- R

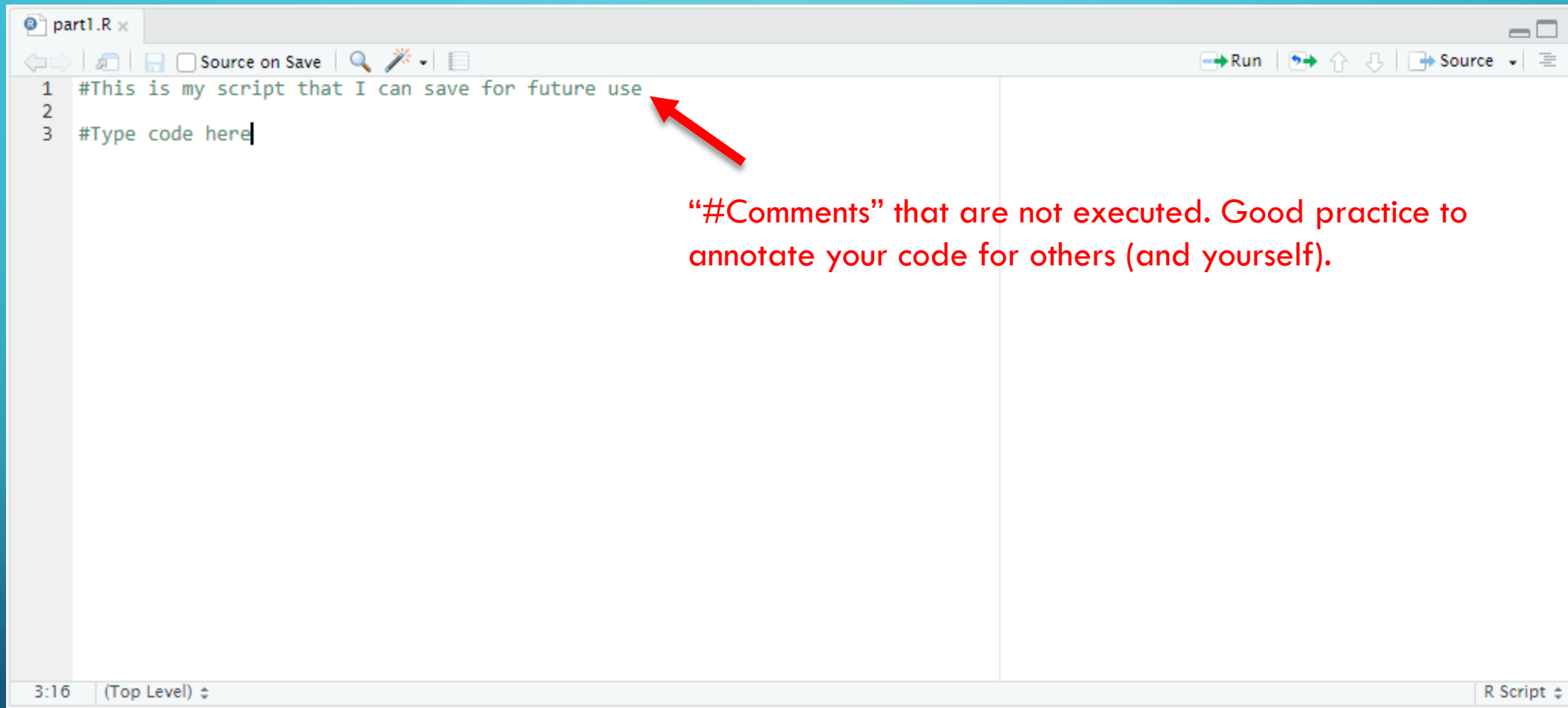
New Folder Choose Cancel

R SCRIPTS

File > New File > R Script



R SCRIPTS



The screenshot shows an R script editor window titled "part1.R". The script contains three lines of code:

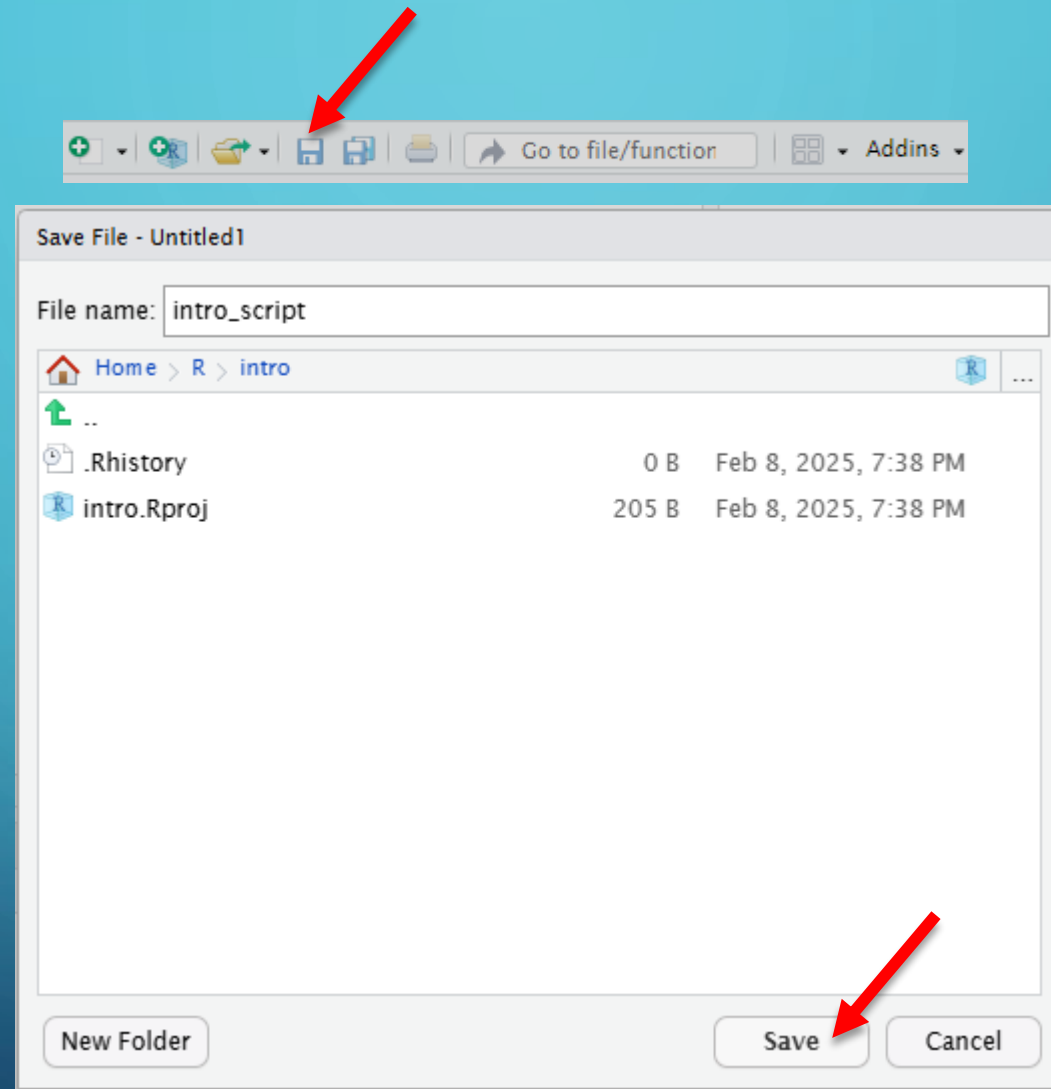
```
1 #This is my script that I can save for future use
2
3 #Type code here|
```

A red arrow points to the first line, which is a comment. To the right of the script, there is a red text annotation: "#Comments" that are not executed. Good practice to annotate your code for others (and yourself).

The editor window includes a toolbar with icons for undo, redo, save, and a "Source on Save" checkbox. On the right side of the toolbar, there are buttons for "Run", "Source", and a dropdown menu. The status bar at the bottom shows the time "3:16", the location "(Top Level)", and the file type "R Script".

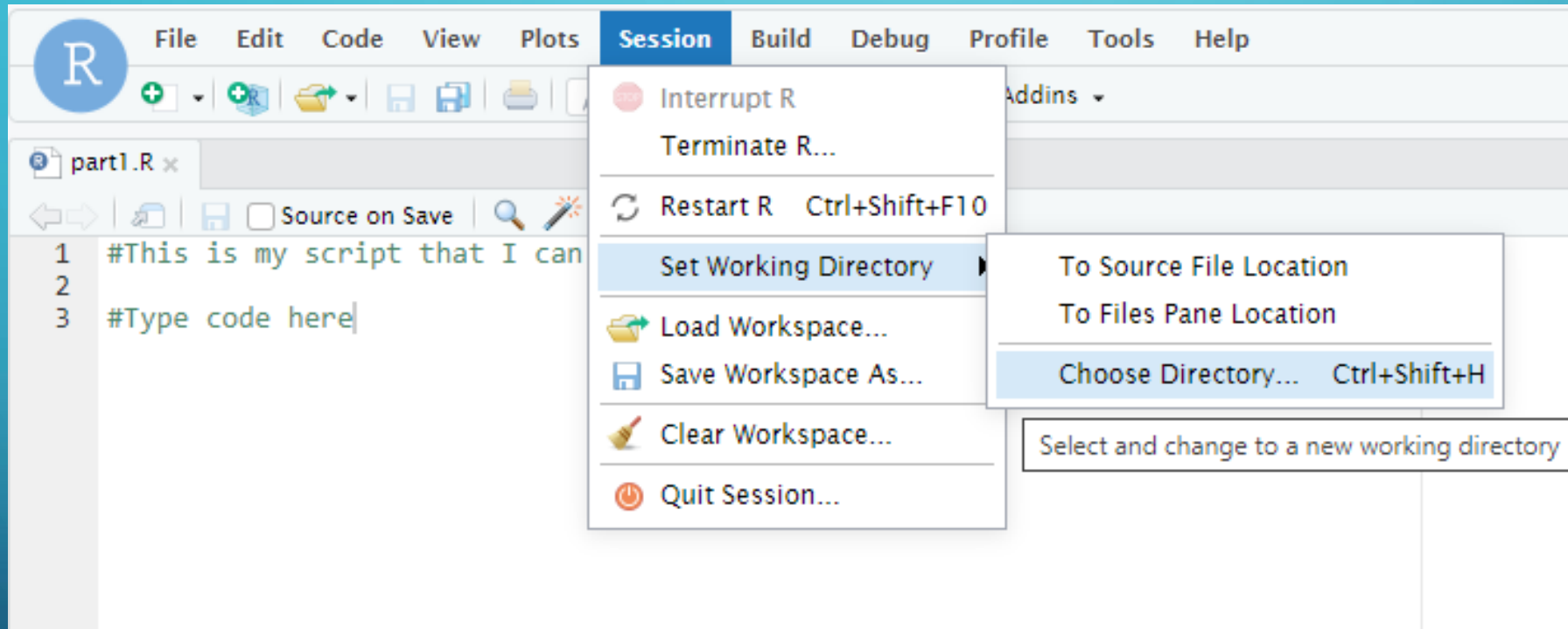
- Scripts can be saved and used again
- Don't start from scratch every time – refer back to/modify your scripts

SAVE YOUR SCRIPT



SETTING THE WORKING DIRECTORY

Session > Set Working Directory > Choose Directory...



*The working directory was set when we created our project

- Check current working directory using console: `getwd()`
- Can also change working directory via console: `setwd("path/path...")`

EXECUTING FROM SCRIPTS

The screenshot shows the RStudio interface with a script editor and a console. The script editor contains the following code:

```
1 # My first script
2
3 1+1
4 |
```

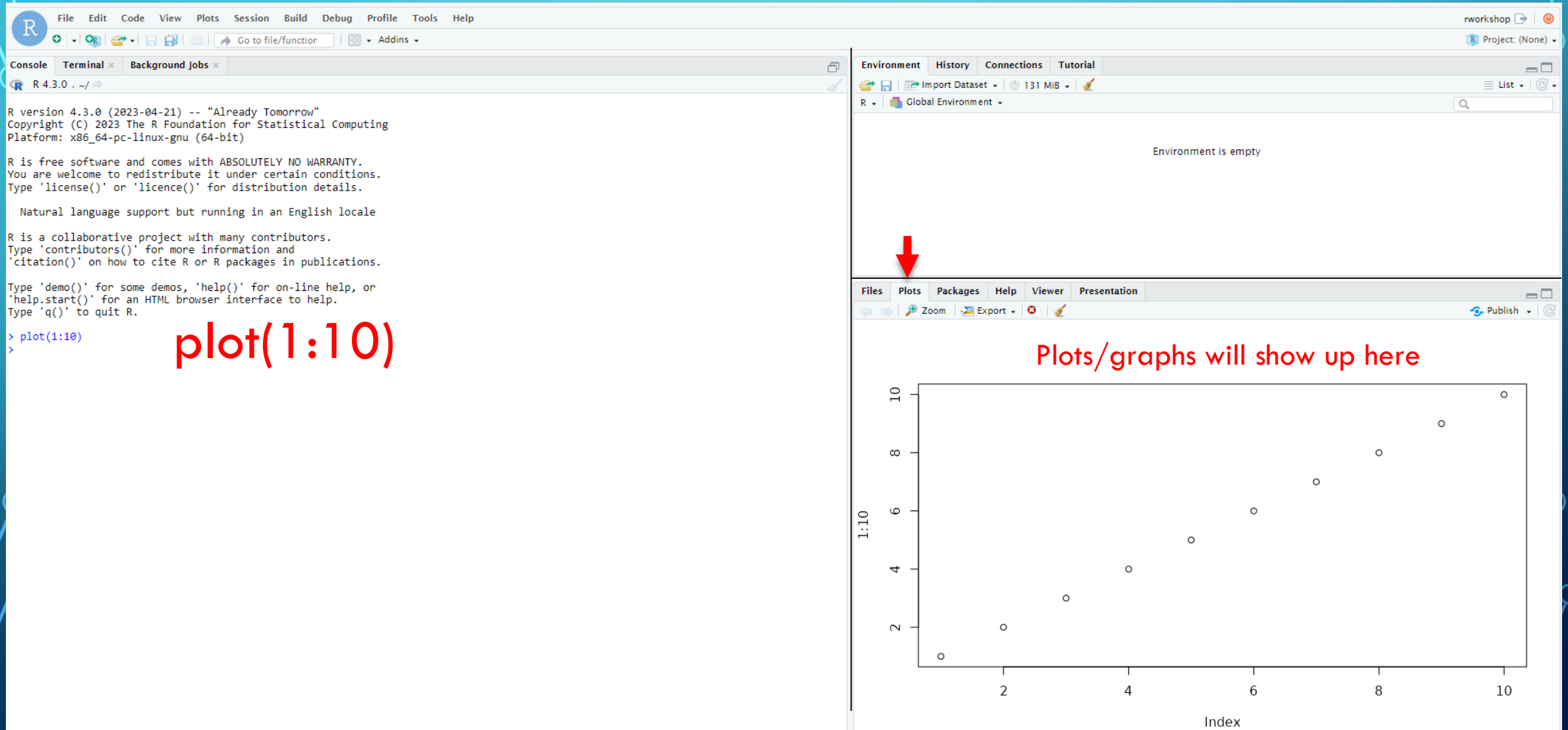
The console shows the output of the script:

```
> 1+1
[1] 2
>
```

Red arrows and text boxes provide instructions on how to execute the code:

- A red box contains the text: "Code can be run directly from your script".
- Below this box is a list of instructions:
 1. Use the run button (top right)
 2. Use the ctrl + Enter keyboard shortcut
- Below the list is a note: "*Will run line or block of code at your current cursor location".
- A red arrow points from the "Run" button in the top right of the script editor to the red box.
- A red arrow points from the console output "[1] 2" to a red box containing the text: "Shows up in console".

PLOT WINDOW



The screenshot displays the RStudio environment. On the left, the console shows the R version 4.3.0 (2023-04-21) and the command `plot(1:10)` entered. On the right, the 'Plots' pane shows a scatter plot of the sequence 1 to 10. A red arrow points to the 'Plots' tab, and a red text label indicates that plots will show up there.

Console Output:

```
R version 4.3.0 (2023-04-21) -- "Already Tomorrow"
Copyright (C) 2023 The R Foundation for Statistical Computing
Platform: x86_64-pc-linux-gnu (64-bit)

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

Natural language support but running in an English locale

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

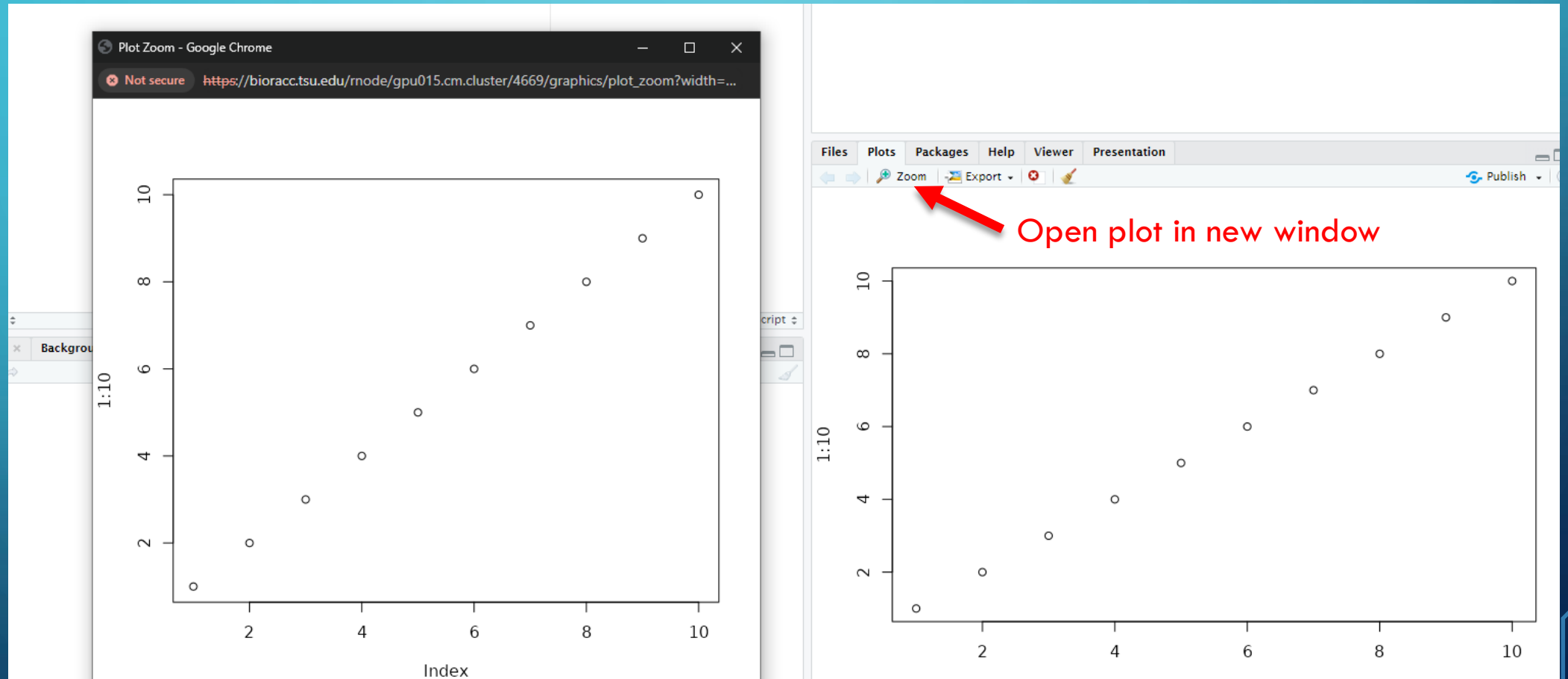
> plot(1:10)
>
```

Plot Window:

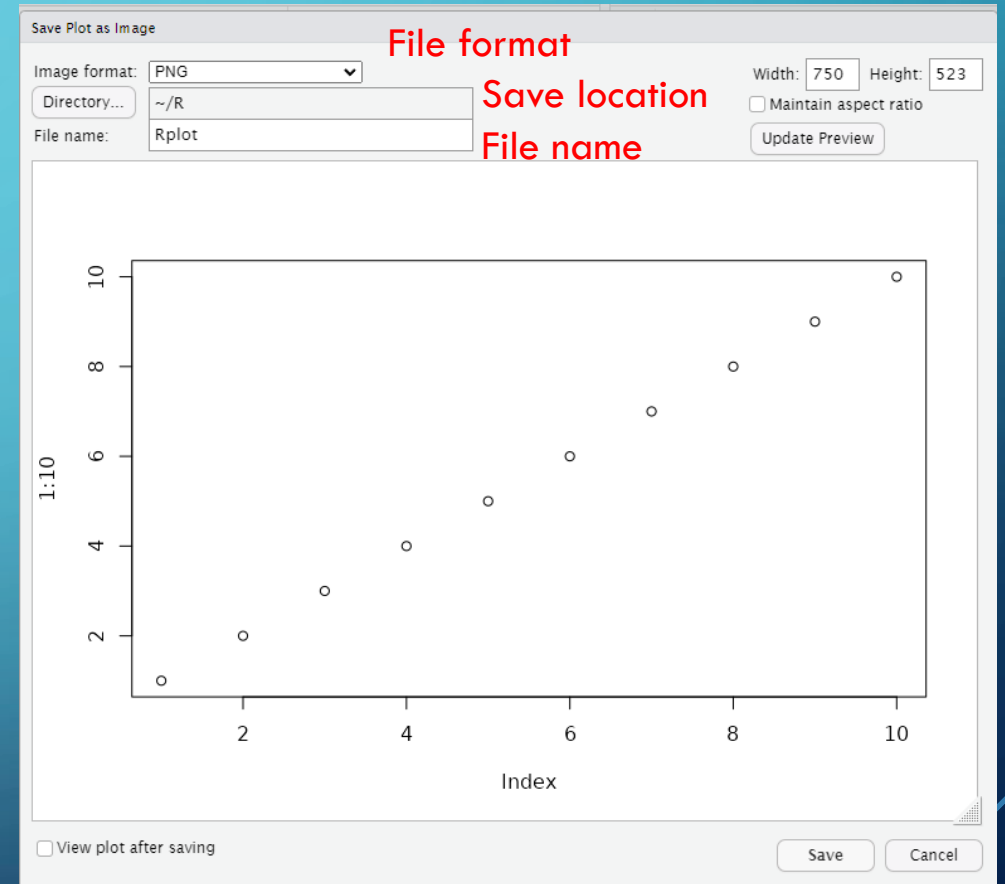
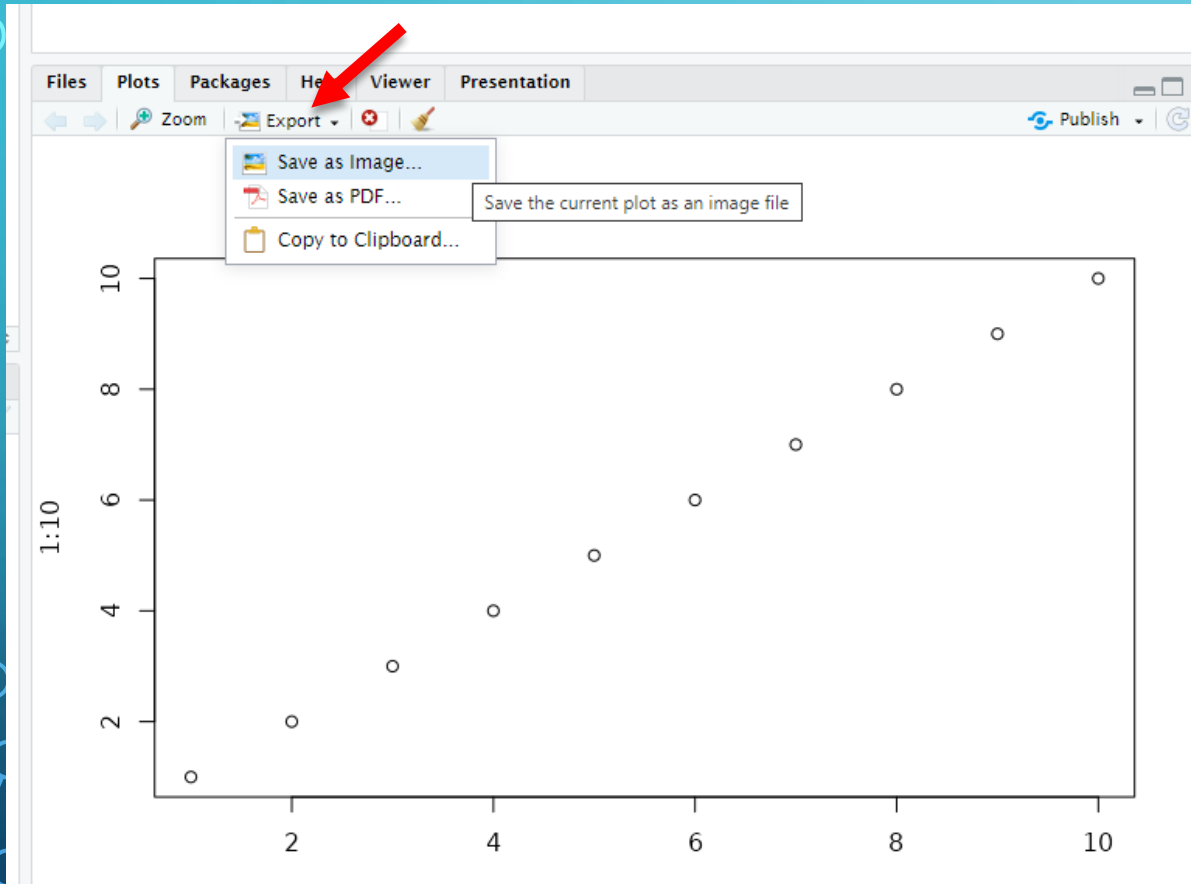
Plots/graphs will show up here

Index	Value
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10

PLOT WINDOW



SAVING PLOTS

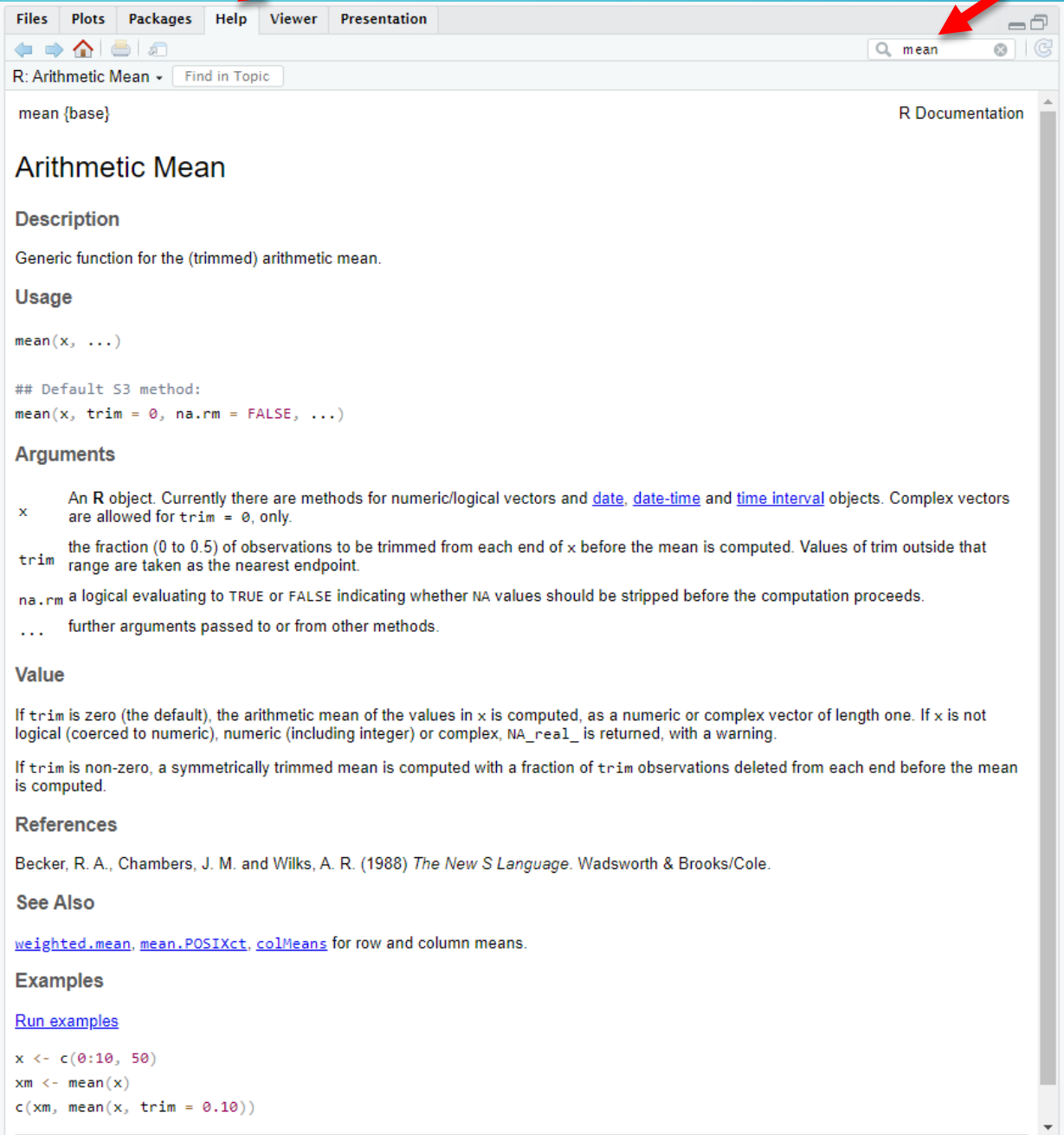


SEARCH IN “HELP” FOR FUNCTIONS

Usage →

Arguments →

Examples →



The screenshot shows the R Help window with the 'mean' function documentation. The 'Help' menu is highlighted with a red arrow, and the search bar in the top right corner contains the text 'mean', also highlighted with a red arrow. The documentation includes the following sections:

- mean {base}**
- Arithmetic Mean**
- Description**
Generic function for the (trimmed) arithmetic mean.
- Usage**
`mean(x, ...)`

Default S3 method:
`mean(x, trim = 0, na.rm = FALSE, ...)`
- Arguments**
 - x** An R object. Currently there are methods for numeric/logical vectors and [date](#), [date-time](#) and [time interval](#) objects. Complex vectors are allowed for `trim = 0`, only.
 - trim** the fraction (0 to 0.5) of observations to be trimmed from each end of `x` before the mean is computed. Values of `trim` outside that range are taken as the nearest endpoint.
 - na.rm** a logical evaluating to TRUE or FALSE indicating whether NA values should be stripped before the computation proceeds.
 - ...** further arguments passed to or from other methods.
- Value**

If `trim` is zero (the default), the arithmetic mean of the values in `x` is computed, as a numeric or complex vector of length one. If `x` is not logical (coerced to numeric), numeric (including integer) or complex, `NA_real_` is returned, with a warning.

If `trim` is non-zero, a symmetrically trimmed mean is computed with a fraction of `trim` observations deleted from each end before the mean is computed.
- References**

Becker, R. A., Chambers, J. M. and Wilks, A. R. (1988) *The New S Language*. Wadsworth & Brooks/Cole.
- See Also**

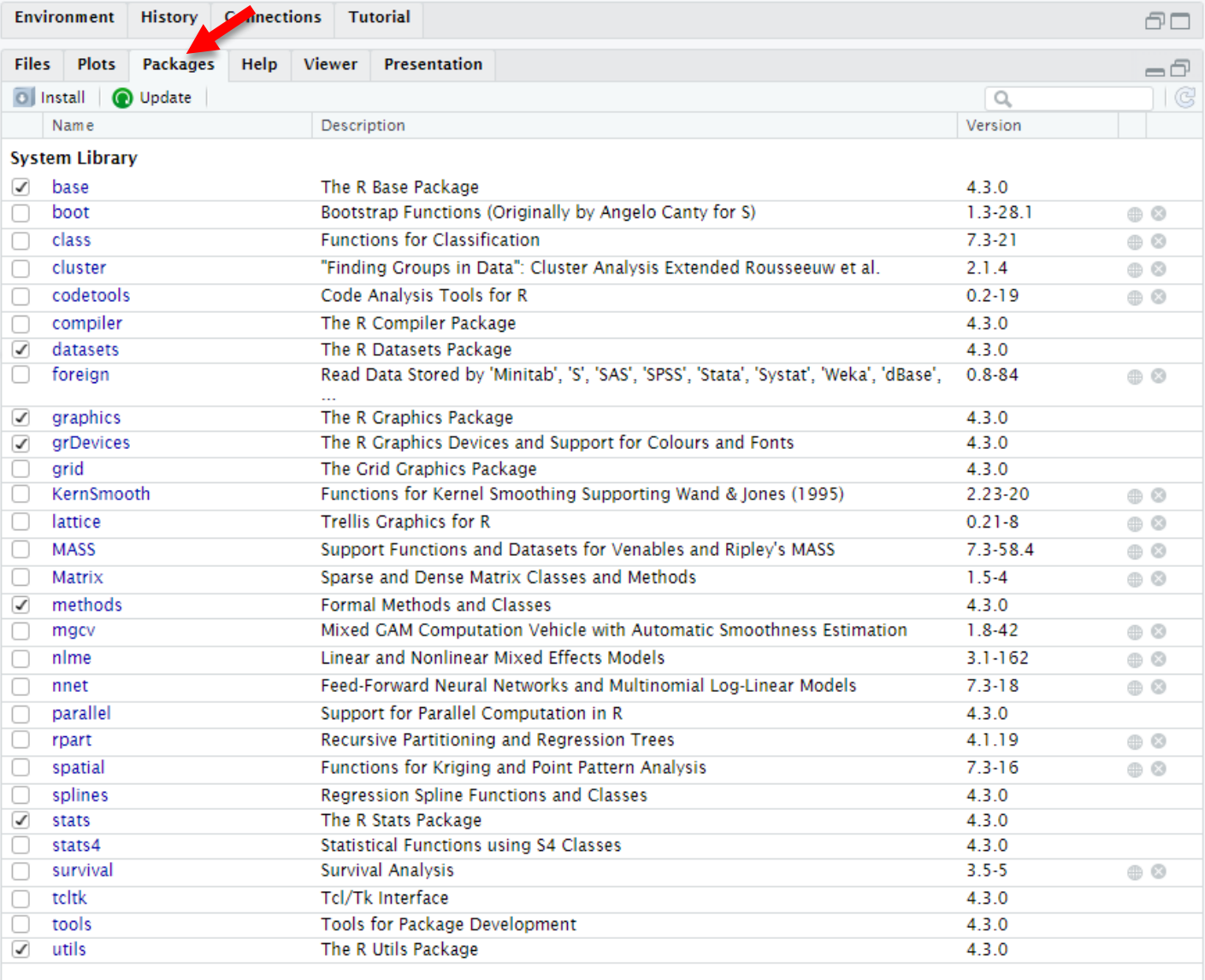
[weighted.mean](#), [mean.POSIXct](#), [colMeans](#) for row and column means.
- Examples**

[Run examples](#)


```
x <- c(0:10, 50)
xm <- mean(x)
c(xm, mean(x, trim = 0.10))
```

R PACKAGES

- Default packages available in System Library
- Check mark shows they are loaded and ready to use
- Can load packages here by checking, or in console
- Packages installed by user will be in User Library



The screenshot shows the RStudio interface with the 'Packages' tab selected. The 'System Library' section lists various R packages. The 'base' package is checked, indicating it is loaded. The 'datasets' package is also checked. The 'foreign' package is not checked. The 'graphics' package is checked. The 'grDevices' package is checked. The 'grid' package is not checked. The 'KernSmooth' package is not checked. The 'lattice' package is not checked. The 'MASS' package is not checked. The 'Matrix' package is not checked. The 'methods' package is checked. The 'mgcv' package is not checked. The 'nlme' package is not checked. The 'nnet' package is not checked. The 'parallel' package is not checked. The 'rpart' package is not checked. The 'spatial' package is not checked. The 'splines' package is not checked. The 'stats' package is checked. The 'stats4' package is not checked. The 'survival' package is not checked. The 'tcltk' package is not checked. The 'tools' package is not checked. The 'utils' package is checked.

Name	Description	Version
System Library		
☒ base	The R Base Package	4.3.0
☐ boot	Bootstrap Functions (Originally by Angelo Canty for S)	1.3-28.1
☐ class	Functions for Classification	7.3-21
☐ cluster	"Finding Groups in Data": Cluster Analysis Extended Rousseeuw et al.	2.1.4
☐ codetools	Code Analysis Tools for R	0.2-19
☐ compiler	The R Compiler Package	4.3.0
☒ datasets	The R Datasets Package	4.3.0
☐ foreign	Read Data Stored by 'Minitab', 'S', 'SAS', 'SPSS', 'Stata', 'Systat', 'Weka', 'dBase', ...	0.8-84
☒ graphics	The R Graphics Package	4.3.0
☒ grDevices	The R Graphics Devices and Support for Colours and Fonts	4.3.0
☐ grid	The Grid Graphics Package	4.3.0
☐ KernSmooth	Functions for Kernel Smoothing Supporting Wand & Jones (1995)	2.23-20
☐ lattice	Trellis Graphics for R	0.21-8
☐ MASS	Support Functions and Datasets for Venables and Ripley's MASS	7.3-58.4
☐ Matrix	Sparse and Dense Matrix Classes and Methods	1.5-4
☒ methods	Formal Methods and Classes	4.3.0
☐ mgcv	Mixed GAM Computation Vehicle with Automatic Smoothness Estimation	1.8-42
☐ nlme	Linear and Nonlinear Mixed Effects Models	3.1-162
☐ nnet	Feed-Forward Neural Networks and Multinomial Log-Linear Models	7.3-18
☐ parallel	Support for Parallel Computation in R	4.3.0
☐ rpart	Recursive Partitioning and Regression Trees	4.1.19
☐ spatial	Functions for Kriging and Point Pattern Analysis	7.3-16
☐ splines	Regression Spline Functions and Classes	4.3.0
☒ stats	The R Stats Package	4.3.0
☐ stats4	Statistical Functions using S4 Classes	4.3.0
☐ survival	Survival Analysis	3.5-5
☐ tcltk	Tcl/Tk Interface	4.3.0
☐ tools	Tools for Package Development	4.3.0
☒ utils	The R Utils Package	4.3.0

WHAT ARE “PACKAGES”?

- A collection of **functions**, data sets, and compiled code that are grouped together to provide specific functionality for a particular task
- Created by users of R (you can create packages!)
- Must be installed and loaded into current R session to utilize

PACKAGES FROM REPOSITORY



Contributed Packages

Available Packages

Currently, the CRAN package repository features 20928 available packages.

[Table of available packages, sorted by date of publication](#)

[Table of available packages, sorted by name](#)

[CRAN Task Views](#) aim to provide some guidance which packages on CRAN are relevant for tasks related to a certain topic. They provide tools to automatically install all packages from each view. Currently, 44 views are available.

Installation of Packages

Please type `help("INSTALL")` or `help("install.packages")` in R for information on how to install packages from this repository. The manual [R Installation and Administration](#) (also contained in the R base sources) explains the process in detail.

Package Check Results

All packages are tested regularly on machines running [Debian GNU/Linux](#), [Fedora](#), macOS (formerly OS X) and Windows.

The results are summarized in the [check summary](#) (some [timings](#) are also available).

Writing Your Own Packages

The manual [Writing R Extensions](#) (also contained in the R base sources) explains how to write new packages and how to contribute them to CRAN.

Repository Policies

The manual [CRAN Repository Policy \[PDF\]](#) describes the policies in place for the CRAN package repository.

Related Directories

[Archive](#)

Previous versions of the packages listed above, and other packages formerly available.

[Orphaned](#)

Packages with no active maintainer, see the corresponding [README](#).

[bin/windows/contrib](#)

Windows binaries of contributed packages

[bin/macosx/contrib](#)

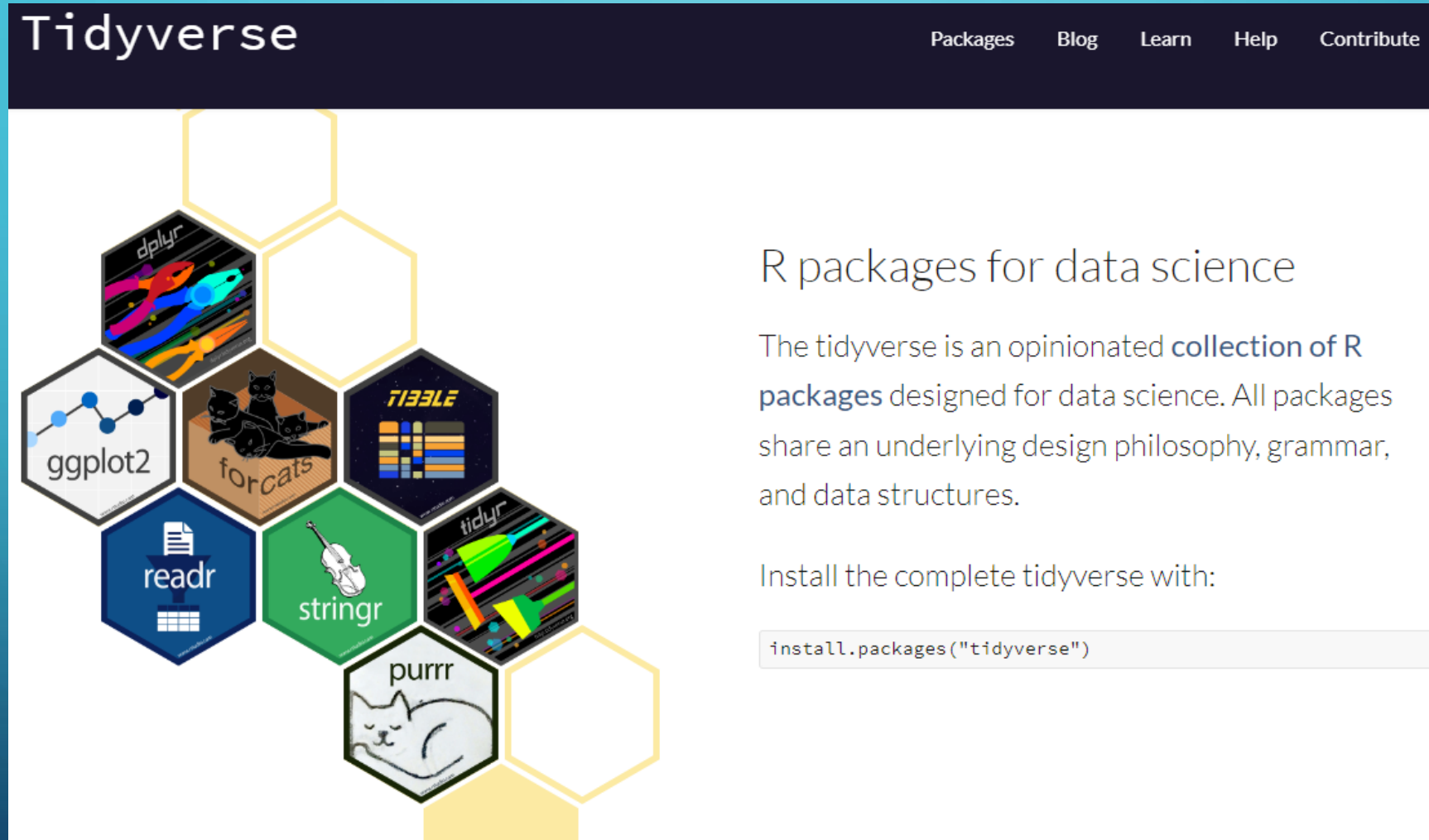
macOS High Sierra binaries of contributed packages

[bin/macosx/el-capitan/contrib](#)

OS X El Capitan binaries of contributed packages

<https://cran.r-project.org/web/packages/>

DATA SCIENCE PACKAGES IN R

The image is a screenshot of the Tidyverse website. At the top, there is a dark blue header with the word "Tidyverse" in white on the left and navigation links "Packages", "Blog", "Learn", "Help", and "Contribute" on the right. Below the header, the main content area has a white background. On the left side of this area, there is a cluster of seven hexagonal icons arranged in a staggered pattern. Each hexagon contains a different icon representing a Tidyverse package: "dplyr" (a colorful abstract shape), "ggplot2" (a blue line graph), "forcats" (three black cats), "readr" (a document icon), "stringr" (a violin), "tidyr" (a colorful abstract shape), and "purrr" (a white cat). To the right of these icons, the text "R packages for data science" is displayed in a large, dark font. Below this, a paragraph explains that the tidyverse is an opinionated collection of R packages designed for data science, sharing a common design philosophy, grammar, and data structures. Further down, the text "Install the complete tidyverse with:" is followed by a code block containing the command `install.packages("tidyverse")`.

Tidyverse

Packages Blog Learn Help Contribute

R packages for data science

The tidyverse is an opinionated **collection of R packages** designed for data science. All packages share an underlying design philosophy, grammar, and data structures.

Install the complete tidyverse with:

```
install.packages("tidyverse")
```

<https://www.tidyverse.org/>

DATA SCIENCE PACKAGES IN R



- Publication-ready figures



- Data manipulation

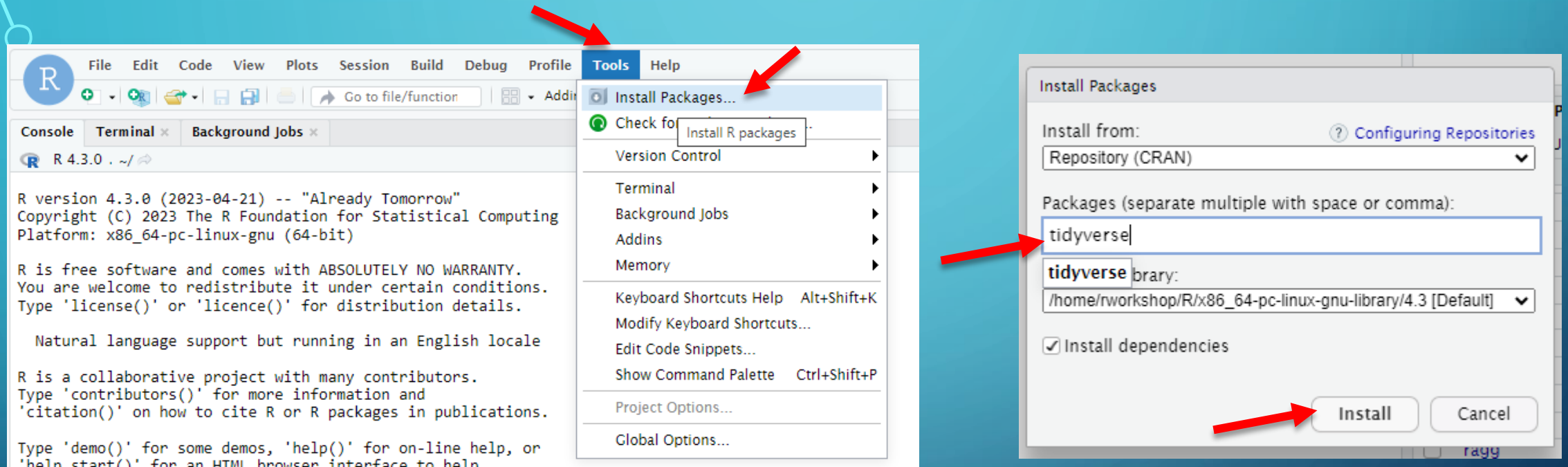


- “tidy” data – standardized way of storing data



- Reading comma- and tab-delimited files (CSV and TSV)

INSTALLING PACKAGES - TIDYVERSE



Do this now as it will take a few minutes
because it is installing several packages



INSTALLING PACKAGES SUCCESSFULLY

```
installing to /home/rworkshop/R/x86_64-pc-linux-gnu-library/4.3/00LOCK-dplyr/00new/dplyr/libs
** R
** data
*** moving datasets to lazyload DB
** inst
** byte-compile and prepare package for lazy loading
** help
*** installing help indices
*** copying figures
** building package indices
** installing vignettes
** testing if installed package can be loaded from temporary location
** checking absolute paths in shared objects and dynamic libraries
** testing if installed package can be loaded from final location
** testing if installed package keeps a record of temporary installation path
* DONE (dplyr)

The downloaded source packages are in
  '/tmp/Rtmp1byGfR/downloaded_packages'
```



```
ERROR: dependency 'curl' is not available for package 'TTR'
* removing '/home/rworkshop/R/x86_64-pc-linux-gnu-library/4.3/TTR'
Warning in install.packages :
  installation of package 'TTR' had non-zero exit status

The downloaded source packages are in
  '/tmp/Rtmp1byGfR/downloaded_packages'
```

BASIC R FUNCTIONS (COMMANDS)

R Reference Card

by Tom Short, EPRI PEAC, tshort@epri-peac.com 2004-11-07

Granted to the public domain. See www.Rpad.org for the source and latest version. Includes material from *R for Beginners* by Emmanuel Paradis (with permission).

Getting help

Most R functions have online documentation.

`help(topic)` documentation on topic

`?topicid`

`help.search("topic")` search the help system

`apropos("topic")` the names of all objects in the search list matching the regular expression "topic"

`help.start()` start the HTML version of help

`str(a)` display the internal *str*ucture of an R object

`summary(a)` gives a "summary" of a, usually a statistical summary but it is *generic* meaning it has different operations for different classes of a

`ls()` show objects in the search path; specify `pat="pat"` to search on a pattern

`ls.str()` `str()` for each variable in the search path

`dir()` show files in the current directory

`methods(a)` shows S3 methods of a

`methods(class=class(a))` lists all the methods to handle objects of class a

Input and output

`load()` load the datasets written with `save`

`data(x)` loads specified data sets

`library(x)` load add-on packages

`read.table(file)` reads a file in table format and creates a data frame from it; the default separator `sep=""` is any whitespace; use `header=TRUE` to read the first line as a header of column names; use `as.is=TRUE` to prevent character vectors from being converted to factors; use `comment.char=""` to prevent "#" from being interpreted as a comment; use `skip=n` to skip n lines before reading data; see the help for options on row naming, NA treatment, and others

`read.csv("filename", header=TRUE)` id. but with defaults set for reading comma-delimited files

`read.delim("filename", header=TRUE)` id. but with defaults set for reading tab-delimited files

`read.fwf(file, widths, header=FALSE, sep=" ", as.is=FALSE)` read a table of fixed width formatted data into a "data.frame"; widths is an integer vector, giving the widths of the fixed-width fields

`save(file,...)` saves the specified objects (...) in the XDR platform-independent binary format

`save.image(file)` saves all objects

`cat(..., file="", sep=" ")` prints the arguments after coercing to

character or factor columns are surrounded by quotes (*); `sep` is the field separator; `eol` is the end-of-line separator; `na` is the string for missing values; use `col.names=NA` to add a blank column header to get the column headers aligned correctly for spreadsheet input

`sink(file)` output to file, until `sink()`

Most of the I/O functions have a `file=""` means the standard input or output. Connections can include files, pipes, zipped files, and R variables.

On windows, the file connection can also be used with `description = "clipboard"`. To read a table copied from Excel, use

`x <- read.delim("clipboard")`

To write a table to the clipboard for Excel, use

`write.table(x, "clipboard", sep="\t", col.names=NA)`

For database interaction, see packages RODBC, DBI, RMySQL, RPostgreSQL, and ROracle. See packages XML, hdf5, netCDF for reading other file formats.

Data creation

`c(...)` generic function to combine arguments with the default forming a vector; with `recursive=TRUE` descends through lists combining all elements into one vector

`from:to` generates a sequence; ":" has operator priority; `1:4 + 1` is "2,3,4,5"

`seq(from,to)` generates a sequence `by=` specifies increment; `length=` specifies desired length

`seq(along=x)` generates 1, 2, ..., `length(along)`; useful for for loops

`rep(x,times)` replicate x times; use `each=` to repeat "each" element of x each times; `rep(c(1,2,3),2)` is 1 2 3 1 2 3; `rep(c(1,2,3),each=2)` is 1 1 2 2 3 3

`data.frame(...)` create a data frame of the named or unnamed arguments; `data.frame(v=1:4, ch=c("a","B","c","d"), n=10)`; shorter vectors are recycled to the length of the longest

`list(...)` create a list of the named or unnamed arguments; `list(a=c(1,2), b="hi", c=31)`

`array(x,dim=)` array with data x; specify dimensions like `dim=c(3,4,2)`; elements of x recycle if x is not long enough

`matrix(x,nrow=,ncol=)` matrix; elements of x recycle

`factor(x,levels=)` encodes a vector x as a factor

`gl(n,k,length=n*k,labels=1:n)` generate levels (factors) by specifying the pattern of their levels; k is the number of levels, and n is the number of replications

`expand.grid()` a data frame from all combinations of the supplied vectors or factors

`rbind(...)` combine arguments by rows for matrices, data frames, and others

`cbind(...)` id. by columns

Slicing and extracting data

Indexing vectors

`x[n]` n^{th} element

`x[-n]` all but the n^{th} element

`x[1:n]` first n elements

Indexing lists

`x[n]` list with elements n

`x[[n]]` n^{th} element of the list

`x[["name"]]` element of the list named "name"

`x$name` id.

Indexing matrices

`x[i,j]` element at row i, column j

`x[i,]` row i

`x[,j]` column j

`x[,c(1,3)]` columns 1 and 3

`x["name",]` row named "name"

Indexing data frames (matrix indexing plus the following)

`x[["name"]]` column named "name"

`x$name` id.

Variable conversion

`as.array(x)`, `as.data.frame(x)`, `as.numeric(x)`,
`as.logical(x)`, `as.complex(x)`, `as.character(x)`,
... convert type; for a complete list, use `methods(as)`

Variable information

`is.na(x)`, `is.null(x)`, `is.array(x)`, `is.data.frame(x)`,
`is.numeric(x)`, `is.complex(x)`, `is.character(x)`,
... test for type; for a complete list, use `methods(is)`

`length(x)` number of elements in x

`dim(x)` Retrieve or set the dimension of an object; `dim(x) <- c(3,2)`

`dimnames(x)` Retrieve or set the dimension names of an object

`nrow(x)` number of rows; `NROW(x)` is the same but treats a vector as a one-row matrix

`ncol(x)` and `NCOL(x)` id. for columns

`class(x)` get or set the class of x; `class(x) <- "myclass"`

`unclass(x)` remove the class attribute of x

`attr(x,which)` get or set the attribute which of x

`attributes(obj)` get or set the list of attributes of obj

Data selection and manipulation

`which.max(x)` returns the index of the greatest element of x

`which.min(x)` returns the index of the smallest element of x

`rev(x)` reverses the elements of x

`sort(x)` sorts the elements of x in increasing order; to sort in decreasing order: `rev(sort(x))`

`cut(x,breaks)` divides x into intervals (factors); `breaks` is the number of cut intervals or a vector of cut points

`match(x, y)` returns a vector of the same length than x with the elements of x which are in y (NA otherwise)

`which(x == a)` returns a vector of the indices of x if the comparison operation is true (TRUE), in this example the values of i for which `x[i] == a` (the argument of this function must be a variable of mode logical)

COMMON FUNCTIONS: C()

- `c()`: concatenate (combine) values into a vector or list
- Default is a vector
- The output type is determined from the highest type of the components in the hierarchy: `NULL < raw < logical < integer < double < complex < character < list` < expression
- Arguably the most used function

```
> a <- c(1, 2, "three")  
> a  
[1] "1"      "2"      "three"
```


ASSIGNING VARIABLES

- Assignment operators:

- Leftward: <-

- Rightward: ->

- Equal: =

Shortcut for leftward:
'Alt + -' on Windows
'Option + -' on Mac

```
> name<-"john"  
> name  
[1] "john"  
> "jane"->name2  
> name2  
[1] "jane"
```

- Use left/rightward for assigning variables (recommended)

- Use = for arguments of functions

```
> X<-seq(10)  
> X  
[1] 1 2 3 4 5 6 7 8 9 10  
> plot(X,)
```

◆ y =	plot.default()
◆ type =	plot.default()
◆ xlim =	plot.default()
◆ ylim =	plot.default()
◆ log =	plot.default()
◆ main =	plot.default()
◆ sub =	plot.default()
◆ xlab =	plot.default()
◆ ylab =	plot.default()
◆ ann =	plot.default()

y
the x and y arguments provide the x and y coordinates for the plot. Any reasonable way of defining the coordinates is acceptable. See the function `xy.coords` for details. If supplied separately, they must be of the same length.
Press F1 for additional help

R DATA TYPES

- Character – “abc”
- Numeric – 3.14
- Integer – 3L (no decimal points)
 - (the “L” declares it as an integer)
- Logical – TRUE
- Factor – “male” (categorical)

```
Console Terminal x Background Jobs x
R 4.3.0 . ~/ ↵
> class("abc")
[1] "character"
> class(3.14)
[1] "numeric"
> class(3L)
[1] "integer"
> class(TRUE)
[1] "logical"
> class(factor("male"))
[1] "factor"
> |
```

R DATA STRUCTURES

Data Structure	Function	Data Type	Dimensions
Vector	c, seq, :	Homogeneous	1
List	list	Heterogeneous	1
Matrix	matrix	Homogeneous	2
Data Frame	data.frame	Heterogeneous	2
Array	array	Homogeneous	N

VECTORS

- Numeric vector

- `numbers <- c(1, 2, 3, 4, 5)`

- Character vector

- `characters <- c("one", "two", "three", "four", "five")`

- Logical vector

- `logicals <- c(TRUE, FALSE, TRUE, FALSE, TRUE)`

```
Console Terminal x Background Jobs x
R 4.3.0 . ~/
> numbers <- c(1, 2, 3, 4, 5)
> numbers
[1] 1 2 3 4 5
>
> characters <- c("one", "two", "three", "four", "five")
> characters
[1] "one" "two" "three" "four" "five"
>
> logicals <- c(TRUE, FALSE, TRUE, FALSE, TRUE)
> logicals
[1] TRUE FALSE TRUE FALSE TRUE
```

MATRICES

Matrix	matrix	Homogeneous	2
--------	--------	-------------	---

- `cells <- c(1, 2, 3, 4)`

`rnames <- c("R1", "R2")`

`cnames <- c("C1", "C2")`

`mymatrix <- matrix(cells, nrow=2, ncol=2, byrow=TRUE, dimnames=list(rnames, cnames))`

Fill row-wise

Row and column names, respectively

```
> cells <- c(1,2,3,4)
> rnames <- c("R1", "R2")
> cnames <- c("C1", "C2")
> mymatrix <- matrix(cells, nrow = 2, ncol = 2, byrow = TRUE, dimnames=list(rnames, cnames))
> mymatrix
  C1 C2
R1 1  2
R2 3  4
```

DATA FRAMES

Data Frame	data.frame	Heterogeneous	2
------------	------------	---------------	---

- `wave <- c(700, 520, 475, 200)`

`light <- c("Red", "Green", "Blue", "Ultraviolet")`

`vis <- c(TRUE, TRUE, TRUE, FALSE)`

`mydf <- data.frame(wave, light, vis)`

`names(mydf) <- c("Wavelength", "Color", "Visible")`

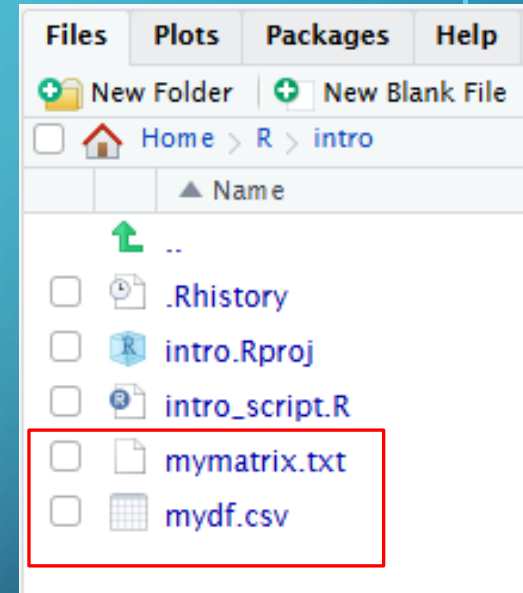
```
> wave <- c(700, 520, 475, 200)
> light <- c("Red", "Green", "Blue", "Ultraviolet")
> vis <- c(TRUE, TRUE, TRUE, FALSE)
> mydf <- data.frame(wave, light, vis)
> names(mydf) <- c("Wavelength", "Color", "Visible")
> mydf
  Wavelength      Color Visible
1         700        Red    TRUE
2         520       Green    TRUE
3         475        Blue    TRUE
4         200 Ultraviolet FALSE
```

EXPORTING (SAVING) FILES

Separator (tab in this case)

- write.table() function
 - write.table(mymatrix, "mymatrix.txt", sep="\t", quote=FALSE)
- write.csv() function
 - write.csv(mydf, "mydf.csv", row.names=FALSE, quote=FALSE)

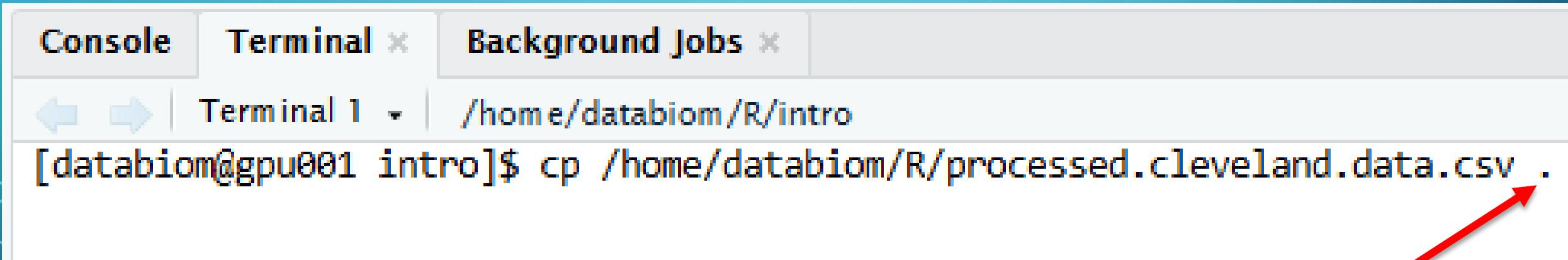
Whether to put quotes around column and row names



```
> write.table(mymatrix, "mymatrix.txt", sep = "\t", quote = FALSE)
> write.csv(mydf, "mydf.csv", row.names = FALSE, quote=FALSE)
```

READING FILES

First, let's copy over an example file to your folder via the **terminal** in RStudio:



The screenshot shows the RStudio interface with three tabs: 'Console', 'Terminal', and 'Background Jobs'. The 'Terminal' tab is active, showing 'Terminal 1' with the current directory set to '/home/databiom/R/intro'. The command entered in the terminal is `cp /home/databiom/R/processed.cleveland.data.csv .`. A red arrow points from a text box to the period at the end of the command.

```
Console | Terminal x | Background Jobs x
Terminal 1 | /home/databiom/R/intro
[databiom@gpu001 intro]$ cp /home/databiom/R/processed.cleveland.data.csv .
```

Don't forget the "."

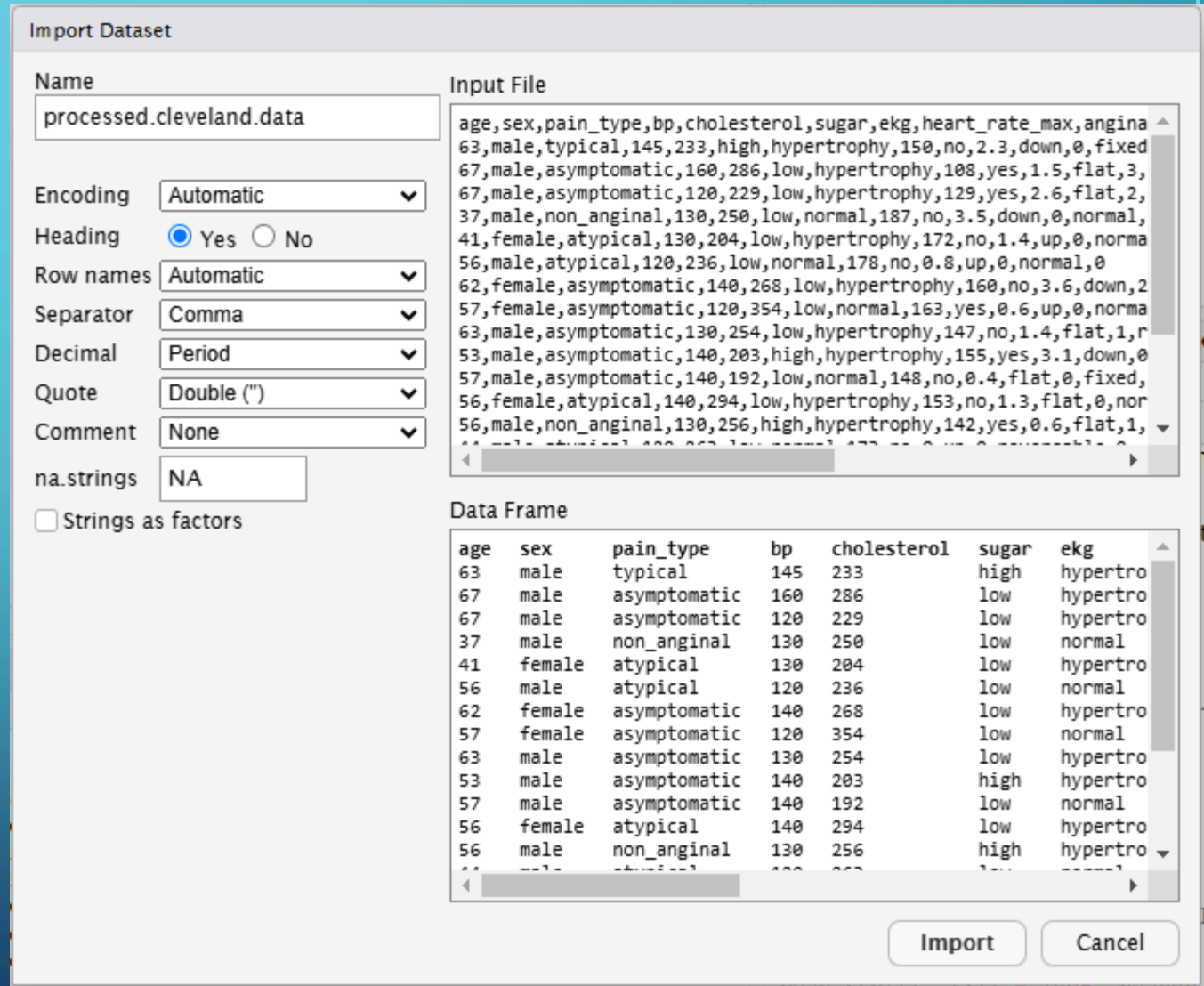
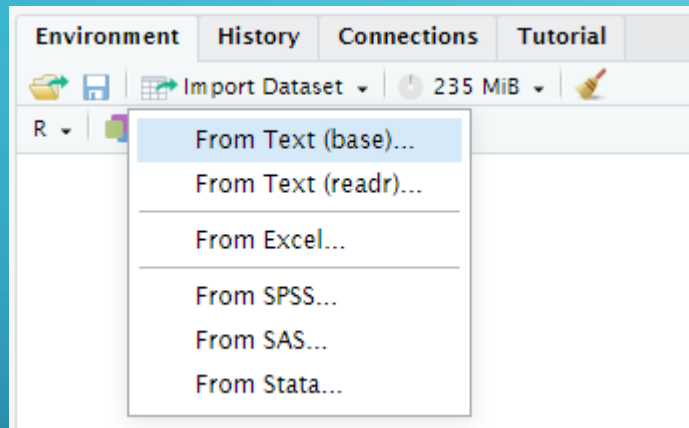
READING CSV FILES USING READ.CSV FUNCTION

```
intro_script.R* x
← → | 📁 | 💾 ☐ Source on Save | 🔍 ✨ | 📋
1 # My first script
2
3 1+1
4
5 mycsv <- read.csv("processed.cleveland.data.csv")
6 head(mycsv)
7
```

```
Console | Terminal x | Background Jobs x
R 4.3.0 . ~/R/intro/ ↗
> mycsv <- read.csv("processed.cleveland.data.csv")
> head(mycsv)
```

	age	sex	pain_type	bp	cholesterol	sugar	ekg	heart_rate_max	angina	stdepression	slope	vessels	defect
1	63	male	typical	145	233	high	hypertrophy	150	no	2.3	down	0	fixed
2	67	male	asymptomatic	160	286	low	hypertrophy	108	yes	1.5	flat	3	normal
3	67	male	asymptomatic	120	229	low	hypertrophy	129	yes	2.6	flat	2	reversible

USING IMPORT DATASET BUTTON



USING THE READ_CSV FUNCTION (READR PACKAGE)

```
library(tidyverse) ← Load tidyverse package
mycsv <- read_csv("processed.cleveland.data.csv")
head(mycsv)
```

```
> head(mycsv)
# A tibble: 6 × 14
  age sex    pain_type    bp cholesterol sugar ekg    heart_rate_max angina stdepression slope vessels defect disease_severity
  <dbl> <chr> <chr>    <dbl>    <dbl> <chr> <chr>    <dbl> <chr>    <dbl> <chr>    <dbl> <chr>    <dbl>
1   63 male  typical    145      233 high hype...    150 no      2.3 down      0 fixed      0
2   67 male  asymptoma... 160      286 low  hype...    108 yes    1.5 flat      3 normal      2
```

SUMMARY STATISTICS

```
31 #Summary stats
32 summary(mycsv)
```

Console Terminal × Background Jobs ×						
R 4.3.0 · ~/R/intro/ ↗						
age	sex	pain_type	bp	cholesterol	sugar	ekg
Min. :29.00	Length:303	Length:303	Min. : 94.0	Min. :126.0	Length:303	Length:303
1st Qu.:48.00	Class :character	Class :character	1st Qu.:120.0	1st Qu.:211.0	Class :character	Class :character
Median :56.00	Mode :character	Mode :character	Median :130.0	Median :241.0	Mode :character	Mode :character
Mean :54.44			Mean :131.7	Mean :246.7		
3rd Qu.:61.00			3rd Qu.:140.0	3rd Qu.:275.0		
Max. :77.00			Max. :200.0	Max. :564.0		
heart_rate_max	angina	stdepression	slope	vessels	defect	disease_severity
Min. : 71.0	Length:303	Min. :0.00	Length:303	Min. :0.0000	Length:303	Min. :0.0000
1st Qu.:133.5	Class :character	1st Qu.:0.00	Class :character	1st Qu.:0.0000	Class :character	1st Qu.:0.0000
Median :153.0	Mode :character	Median :0.80	Mode :character	Median :0.0000	Mode :character	Median :0.0000
Mean :149.6		Mean :1.04		Mean :0.6722		Mean :0.9373
3rd Qu.:166.0		3rd Qu.:1.60		3rd Qu.:1.0000		3rd Qu.:2.0000
Max. :202.0		Max. :6.20		Max. :3.0000		Max. :4.0000
				NA's :4		

THE PIPE OPERATOR: %>%

- The pipe operator allows for “chaining” several actions together
- Reduces the need for intermediate variable creation
- Implemented in Tidyverse

Shortcut:
ctrl + Shift + m

```
34 #Using the pipe operator ( %>% )  
35 mycsv %>% summary()  
36
```

DROPPING MISSING VALUES (NA)

- `drop_na()` function will drop **all rows** that contain NAs unless otherwise specified
- We can use the pipe operator to first drop NAs, then get the summary stats, **all in one line of code**

```
37 #Drop missing values and get summary stats
38 mycsv %>% drop_na() %>% summary()
```

MODIFYING DATASET AND CREATING NEW VARIABLE

- When we modify a dataset, such as removing missing values, we aren't editing the original object/variable
- We must create a new variable to keep changes made

```
40 #Drop NAs and create a new variable
41 mycsv_noNAs <- mycsv %>% drop_na()
42 mycsv_noNAs
```

```
> mycsv_noNAs
# A tibble: 297 × 14
```

Original dataset had 303 rows



DATA WRANGLING WITH DPLYR



- There are a lot more functions for “data wrangling”
 - Filtering
 - Extracting
 - Mutating
 - Grouping
- We will be using bioinformatics packages that handle most tasks, but the important thing is **importing data** for analyses

BIOINFORMATICS PACKAGES

[About](#)[Learn](#)[Packages](#)[Developers](#)[Get Started >](#)

Open source software for **Bioinformatics**

The Bioconductor project aims to develop and share open source software for precise and repeatable analysis of biological data.
We foster an inclusive and collaborative community of developers and data scientists.

INSTALL BIOCONDUCTOR

Install Packages

Install from: [? Configuring Repositories](#)

Repository (CRAN) ▼

Packages (separate multiple with space or comma):

BiocManager

Install to Library:

/home/databiom/R/x86_64-pc-linux-gnu-library/4.3 [Default] ▼

☒ Install dependencies

Install Cancel



INSTALL THE DESEQ2 PACKAGE

```
45 #Install DESeq2 Package via BiocManager  
46 BiocManager::install("DESeq2")
```



After the installation finishes, test that you can load the package:

```
48 library(DESeq2)
```

GOOD JOB!

We'll be using RStudio for “downstream” analyses, so please practice using it!



Don't forget to save your script.

